ACADEMIC CAPSTONE PROJECT REPORT

ON

AI-POWERED SMART ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION (SAMS-FR)

Submitted in partial fulfillment of the requirements for the award of the degree of
Bachelor of Technology
in
Computer Science and Engineering

SUBMITTED BY:	SUPERVISED BY:
[Candidate Name 1] (Roll No: [XXXXX])	[Supervisor Name]
[Candidate Name 2] (Roll No: [XXXXX])	[Designation / Professor]
[Candidate Name 3] (Roll No: [XXXXX])	Department of Computer Science & Engg.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
[INSTITUTION / UNIVERSITY NAME]
[CAMPUS LOCATION, CITY, STATE - PIN CODE]

ACADEMIC YEAR: 2026–2027

CERTIFICATE OF APPROVAL

This is to certify that the capstone project report entitled "AI-POWERED SMART ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION (SAMS-FR)" submitted by [Candidate Name(s)] (Enrollment No(s): [XXXXX]) in partial fulfillment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering from [Institution / University Name] is an authentic record of bonafide research and engineering carried out under my supervision and guidance.

To the best of my knowledge, the matter embodied in this report has not been submitted to any other University or Institute for the award of any degree, diploma, or academic credit.

[Supervisor Name]	[Coordinator Name]	[Head of Department]
Project Guide	Project Coordinator	Head of Department
Department of CSE	Department of CSE	Department of CSE

CANDIDATE DECLARATION

I/We hereby declare that the work presented in this project report entitled "AI-POWERED SMART ATTENDANCE MANAGEMENT SYSTEM USING FACE RECOGNITION (SAMS-FR)" is an authentic record of our own investigative research, software engineering, and experimental validation conducted under the supervision of [Supervisor Name], [Designation], Department of Computer Science and Engineering, [Institution / University Name].

I/We confirm that this report has not formed the basis for the award of any other academic degree, diploma, fellowship, or similar title at any other institution. All contributions, external technical libraries, open-source algorithms, and documented methodologies have been cited in accordance with academic standards.

Date: [DD/MM/YYYY]
Place: [City, State]

_____
[Candidate Name(s)]
Enrollment No(s): [XXXXX]
Department of Computer Science & Engg.

ACKNOWLEDGEMENT

We convey our profound gratitude and heartfelt thanks to our project supervisor, [Supervisor Name], [Designation], Department of Computer Science and Engineering, for their invaluable academic guidance, constructive feedback, technical mentorship, and patient encouragement throughout every stage of this project.

We express our sincere appreciation to [Head of Department Name], Head of the Department of Computer Science and Engineering, for making available the necessary high-performance computing facilities, laboratory infrastructure, and institutional support required for software development and automated testing.

We also thank the faculty members, laboratory instructors, and administrative staff of the Department of Computer Science and Engineering for their direct and indirect assistance. Special thanks are extended to our classmates and peers who volunteered to participate in testing and evaluating the facial recognition pipeline.

Finally, we express our heartfelt appreciation to our parents and family members whose continuous encouragement, patience, and moral support provided the cornerstone of our academic journey.

ABSTRACT

Attendance management in modern educational institutions is an essential operational activity that directly underpins student academic monitoring, regulatory compliance, and institutional discipline. Conventional attendance tracking mechanisms—predominantly manual paper-based roll calls and shared hardware biometric devices (such as optical fingerprint scanners or dedicated facial terminals)—suffer from profound operational bottlenecks. Manual methods consume 10 to 15 minutes of valuable classroom instructional time, are highly susceptible to human recording errors, and remain severely vulnerable to proxy attendance. Conversely, dedicated hardware biometric stations necessitate significant capital investment, suffer from long physical queues before and after class periods, cause class delays when shared devices are misplaced or undergoing maintenance, and carry physical hygiene concerns.

To resolve these critical systemic challenges, this project presents the design, full-stack implementation, and empirical validation of the AI-Powered Smart Attendance Management System using Face Recognition (SAMS-FR). SAMS-FR is an enterprise-grade, secure, web-based platform that democratizes biometric attendance recording by decoupling face recognition from proprietary physical devices. By leveraging standard web browsers and device cameras on faculty laptops, desktop terminals, or tablets, authorized instructors can mark attendance in real time through an automated, contactless, computer-vision-driven workflow.

The system is engineered on a robust three-tier architecture: a modern responsive Single-Page Application (SPA) frontend developed with React.js and Vite; an asynchronous application service layer constructed with Python and the Flask framework; and a high-performance relational database layer utilizing MySQL Server with SQLAlchemy 2.0 ORM. The core biometric engine integrates OpenCV for image pipeline preprocessing and dlib's deep residual network (ResNet-34) to extract 128-dimensional facial embedding vectors. Cosine similarity and Euclidean distance thresholding compare detected faces against enrolled student profiles with sub-second execution speeds. An integrated anti-spoofing mechanism utilizing MediaPipe Face Mesh landmark tracking monitors passive liveness to thwart photo-attack and video replay fraud.

SAMS-FR delivers three fully isolated, role-specific portals: an Administrator Portal for holistic curriculum, timetable, user, and biometric lifecycle management; a Faculty Portal

for real-time attendance sessions, coursework assignment creation, study material distribution, and gradebook moderation; and a Student Portal for transparent attendance history tracking, subject-wise analytics against the statutory 75% threshold, study material downloads, and academic transcripts. Comprehensive empirical testing across 486 automated test cases demonstrates 100% test pass fidelity, sub-second face identification latency, zero ESLint defects, and a production build time of 2.04 seconds. The system demonstrates an 85% reduction in attendance recording time, eliminates proxy attendance, and establishes a secure, scalable foundation for higher educational institutions.

Keywords: Keywords: Face Recognition, Smart Attendance, Artificial Intelligence, Computer Vision, Deep Learning, Flask, React.js, MySQL, Role-Based Access Control, Contactless Biometrics, Anti-Spoofing, Academic Management.

TABLE OF CONTENTS

CERTIFICATE OF APPROVAL i

CANDIDATE DECLARATION ii

ACKNOWLEDGEMENT iii

ABSTRACT iv

TABLE OF CONTENTS vi

LIST OF FIGURES x

LIST OF TABLES xi

LIST OF ABBREVIATIONS xii

CHAPTER 1: INTRODUCTION 1

1.1 Introduction 1

1.2 Background 1

1.3 Problem Statement 2

1.4 Project Objectives 2

1.5 Project Scope 3

1.6 Proposed Solution 4

1.7 Expected Outcomes 4

1.8 Organization of the Report 5

CHAPTER 2: LITERATURE REVIEW 6

2.1 Introduction 6

2.2 Existing Attendance Methods 6

2.3 Biometric Attendance Systems 6

2.4 Face Detection and Recognition 6

2.5 Existing System Limitations 7

2.6 Proposed System Advantages 7

2.7 Summary 8

CHAPTER 3: REQUIREMENTS ANALYSIS 8

3.1 Functional Requirements 8
3.2 Non-Functional Requirements 9
3.3 Admin Requirements 9
3.4 Faculty Requirements 9
3.5 Student Requirements 9
3.6 Hardware Requirements 10
3.7 Software Requirements 10
3.8 Feasibility Analysis 11
3.9 Development Methodology: Agile Model 11
CHAPTER 4: SYSTEM DESIGN 13
4.1 System Architecture 13
4.2 System Modules 14
4.3 Data Flow Diagram Level 0 15
4.4 Data Flow Diagram Level 1 16
4.5 Use Case Diagram 18
4.6 Entity Relationship Diagram 20
4.7 Database Design 20
4.8 Face Recognition Workflow 21
4.9 Attendance Workflow 21
4.10 Security Design 22
CHAPTER 5: SYSTEM IMPLEMENTATION 22
5.1 Technology Stack 22
5.2 Frontend Implementation 22
5.3 Backend Implementation 22
5.4 Database Implementation 22
5.5 Authentication and Authorization 22
5.6 Face Recognition Implementation 23

5.7 Attendance Management 23
5.8 Academic Management 23
5.9 Assignment and Study Material Module 23
5.10 Reporting and Export Engine 23
5.11 API Integration 24
CHAPTER 6: APPLICATION DEMONSTRATION 24
6.1 Authentication Portal 24
6.2 Administrator Portal 25
6.3 Faculty Portal 32
6.4 Student Portal 36
6.5 Important End-to-End Workflows 42
CHAPTER 7: TESTING 42
7.1 Testing Strategy 42
7.2 Unit Testing 42
7.3 Integration Testing 42
7.4 API Testing 42
7.5 Frontend Testing 42
7.6 Build and Static Analysis 42
7.7 Security Testing 43
7.8 Biometric Evaluation 43
7.9 Presentation Attack Detection & Liveness Evaluation 44
7.10 Test Results 45
CHAPTER 8: RESULTS AND DISCUSSION 46
8.1 Implemented Features vs Proposed Synopsis 46
8.2 Functional Results 46
8.3 System Performance 46
8.4 Comparison with Existing Methods 46

8.5 Limitations and Observations 47
CHAPTER 9: CONCLUSION AND FUTURE SCOPE 47
9.1 Conclusion 47
9.2 Project Achievements 48
9.3 Current Limitations 48
9.4 Future Enhancements 48
REFERENCES 50
APPENDICES 51
APPENDIX A: ADDITIONAL APPLICATION SCREENSHOTS AND WORKFLOWS 51
APPENDIX B: API ENDPOINT INVENTORY 59
APPENDIX C: DATABASE TABLES AND RELATIONSHIPS 61
APPENDIX D: INSTALLATION AND SETUP GUIDE 61
APPENDIX E: ADDITIONAL TEST EVIDENCE 61

LIST OF FIGURES

No table of figures entries found.

LIST OF TABLES

No table of figures entries found.

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
APCER	Attack Presentation Classification Error Rate
API	Application Programming Interface
BPCER	Bona Fide Presentation Classification Error Rate
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
CV	Computer Vision
DFD	Data Flow Diagram
DL	Deep Learning
DPI	Dots Per Inch
EAR	Eye Aspect Ratio
ERD	Entity Relationship Diagram
FAR	False Acceptance Rate
FRR	False Rejection Rate
GPA	Grade Point Average
HOG	Histogram of Oriented Gradients
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
JWT	JSON Web Token
LMS	Learning Management System
MySQL	My Structured Query Language

OpenCV	Open Source Computer Vision Library
ORM	Object-Relational Mapping
OS	Operating System
PAD	Presentation Attack Detection
PDF	Portable Document Format
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer
RFID	Radio-Frequency Identification
SAMS-FR	Smart Attendance Management System using Face Recognition
SPA	Single Page Application
SQL	Structured Query Language
SSD	Solid-State Drive
UI	User Interface
UX	User Experience
Vite	Next Generation Frontend Tooling
WSGI	Web Server Gateway Interface

CHAPTER 1: INTRODUCTION

1.1 Introduction

Attendance management is an essential operational activity in educational institutions. It serves as the primary instrument through which student classroom engagement, course progression, and regulatory compliance are tracked. Collegiate governing bodies and university frameworks mandate rigorous attendance tracking to establish examination eligibility, course credit accreditation, scholarship disbursements, and academic standing.

Despite rapid digitalization across higher education workflows, attendance recording remains an operational challenge in many institutions. Systems commonly oscillate between manual paper-based registers and dedicated hardware biometric devices installed at classroom entrances. Both approaches present operational hurdles: manual roll calls consume valuable classroom instructional time and are prone to proxy attendance, whereas dedicated biometric stations require physical hardware procurement, ongoing maintenance, and shared-device logistics.

The Smart Attendance Management System using Face Recognition (SAMS-FR) is designed to address these challenges. SAMS-FR is an AI-powered, web-based platform that decouples biometric attendance recording from dedicated physical hardware. By utilizing standard web browsers and device cameras on faculty laptops, tablets, or mobile devices, SAMS-FR enables authorized faculty members to conduct automated attendance sessions in real time, securely recording data into a centralized relational database.

1.2 Background

Biometric authentication in educational institutions historically relied on physical contact sensors, predominantly optical or capacitive fingerprint readers. While fingerprint systems offered improvements over manual paper ledgers, their practical academic deployment revealed persistent drawbacks: sensor degradation due to dust and skin oils, extended queuing times during short lecture transitions, and physical hygiene concerns.

Facial recognition has emerged as an effective contactless biometric identification method. By analyzing distinct physiological facial geometry and comparing features against pre-registered embeddings, computer vision algorithms achieve verification without physical

contact. However, institutional adoption of facial recognition has frequently been restricted by dedicated hardware appliances. Standalone facial terminals require local installation, power cabling, and isolated networks, preventing seamless integration with central academic curricula, timetables, and student information systems.

SAMS-FR bridges this gap by integrating computer vision models into an open, modern three-tier web application stack. By performing camera capture directly through the web browser and executing neural recognition pipelines in an asynchronous Flask backend, the system achieves accessible multi-device operation.

1.3 Problem Statement

The operational challenges characterizing traditional and semi-automated academic attendance tracking fall into four primary categories:

- **Instructional Time Consumption:** Traditional manual attendance marking via roll calls consumes between 10 and 15 minutes of an instructional hour. In lecture cohorts containing 60 to 100 students, this represents an unrecoverable loss of 15% to 25% of dedicated teaching time.
- **Proxy Attendance and Integrity Vulnerabilities:** Manual paper registers are vulnerable to proxy attendance, where present students mark attendance on behalf of absent peers, compromising the authenticity of institutional records. Paper registers are also vulnerable to physical damage and clerical tally errors.
- **Hardware Dependency and Sharing Delays:** Institutions adopting biometric systems frequently rely on dedicated physical devices. Sharing a limited inventory of physical hardware among multiple instructors causes scheduling delays, device unavailability, battery charging issues, and instructional disruption.
- **Data Fragmentation and Lack of Real-Time Tracking:** Attendance data in legacy systems often remains siloed in offline ledgers or isolated device memories. Consolidating reports requires substantial clerical effort, and students lack real-time visibility into their cumulative attendance standing prior to examination debarment.

1.4 Project Objectives

The primary objective of the SAMS-FR project is to develop a secure, efficient, and automated web-based attendance management system while improving record reliability and reducing administrative overhead. In accordance with Section 3 of the original project synopsis, the specific technical objectives comprise:

- **Objective 1:** To develop a web-based attendance management system that enables authorized faculty members to record attendance using any authenticated device equipped with a camera through a standard web browser.
- **Objective 2:** To implement facial recognition technology for automatically identifying registered students and marking attendance in real time.
- **Objective 3:** To reduce proxy attendance and human errors by using AI-based facial recognition and secure role-based authentication.
- **Objective 4:** To provide separate portals for administrators, faculty members, and students with role-specific functionalities and secure access.
- **Objective 5:** To maintain a centralized database for storing student information, attendance records, faculty details, and academic data.
- **Objective 6:** To generate attendance reports and basic analytics that assist faculty members and administrators in monitoring student attendance efficiently.
- **Objective 7:** To develop a responsive web application that can be accessed from desktops, laptops, tablets, and smartphones without requiring dedicated attendance hardware.
- **Objective 8:** To design a scalable system architecture that can be extended in the future with features such as timetable integration, temporary faculty assignment, anti-spoofing techniques, cloud deployment, and mobile application support.

1.5 Project Scope

The scope of SAMS-FR focuses on providing a centralized platform with role-based access for administrators, faculty members, and students within educational institutions. Faculty members authenticate securely, access assigned classes, and conduct attendance sessions using any authorized camera-equipped device. Administrators manage students, faculty, departments, courses, offerings, and system configurations, while students view their attendance history and compliance percentages.

To maintain academic honesty, the scope is categorized into core implemented deliverables and extended features:

- **Implemented Core Scope:** Core web application, role-based authentication, student and faculty directories, camera-assisted attendance sessions, computer vision face matching, centralized MySQL database storage, and multi-format report exports.
- **Implemented Extended Capabilities:** Features originally identified in the synopsis as future extensions—including timetable management, temporary faculty substitution, anti-spoofing presentation attack detection, coursework distribution, and

academic results moderation—were subsequently developed and validated during later project phases.

- **Future Scope:** Native mobile application binaries (.apk/.ipa), institutional ERP/LMS single-sign-on integration, and distributed multi-campus cloud deployments remain reserved for future enterprise development.

1.6 Proposed Solution

SAMS-FR replaces manual roll calls and shared hardware terminals with a three-tier web application. By utilizing the HTML5 MediaDevices API, the system streams camera frames from standard web browsers to a Flask backend service. The backend coordinates face detection, landmark alignment, deep metric feature extraction, and cosine similarity matching against enrolled student embeddings. Verified attendances are committed atomically to a centralized MySQL database.

1.7 Expected Outcomes

In accordance with Section 17 of the project synopsis, the expected outcomes of SAMS-FR include:

- Automating the attendance marking process using facial recognition technology.
- Reducing manual effort and minimizing errors associated with traditional attendance methods.
- Significantly reducing proxy attendance through AI-based face recognition and secure authentication.
- Providing a centralized platform for managing students, faculty members, subjects, departments, attendance records, and academic information.
- Enabling authorized faculty members to conduct attendance using any authenticated device equipped with a camera through a web browser.
- Providing separate and secure portals for Administrators, Faculty Members, and Students with role-based access control.
- Generating attendance reports and attendance statistics for effective academic monitoring and decision-making.
- Improving transparency by allowing students to access their attendance history and attendance percentage in real time.
- Reducing dependency on dedicated attendance hardware, making the system more flexible and cost-effective.

- Providing a scalable foundation that can support future enhancements such as mobile applications, cloud deployment, timetable integration, anti-spoofing techniques, and ERP integration.

1.8 Organization of the Report

The remainder of this report is organized as follows: Chapter 2 reviews relevant literature and examines existing attendance systems. Chapter 3 provides requirements analysis, hardware/software specifications, and development methodology. Chapter 4 presents system architecture, database design, and UML models. Chapter 5 covers technology stack implementation. Chapter 6 demonstrates application portals with genuine screenshots. Chapter 7 presents testing strategies and empirical validation. Chapter 8 discusses results, operational impacts, and comparisons. Chapter 9 concludes the report and outlines future enhancements.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Academic attendance management has transitioned across several technological paradigms. This chapter evaluates manual roll calls, smart card systems, contact-based biometrics, and dedicated facial terminals to highlight the technological rationale for SAMS-FR.

2.2 Existing Attendance Methods

Manual roll call remains common in educational institutions due to its zero initial equipment cost. Instructors read student names sequentially from a paper register and mark attendance status manually. However, this approach consumes 10 to 15 minutes of lecture time and is vulnerable to vocal proxy marking by peers in large lecture halls. At the end of the term, compiling attendance percentages requires manual calculations that are susceptible to arithmetic mistakes.

2.3 Biometric Attendance Systems

Early automated systems utilized RFID smart cards or optical fingerprint sensors. RFID cards accelerate attendance recording but fail to verify biometric identity, as students can carry and scan cards for absent classmates [1].

Fingerprint scanners verify physical identity using epidermal ridge characteristics. However, physical contact sensors degrade when exposed to moisture or dust, leading to higher False Rejection Rates (FRR) [11]. In collegiate environments where large cohorts transition between classes simultaneously, fingerprint stations create physical queues and delay class commencement.

2.4 Face Detection and Recognition

Facial recognition provides a contactless identification mechanism. Modern pipelines utilize deep Convolutional Neural Networks (CNNs) to extract facial landmarks and map faces into multidimensional embedding spaces [12, 14]. Deep metric architectures, such as FaceNet and dlib's ResNet model, generate compact 128-dimensional vector representations where embeddings of the same individual exhibit minimal distance [15].

Standalone hardware facial recognition kiosks address contact limitations but introduce high equipment expenses. Equipping multiple classrooms with dedicated kiosks is cost-prohibitive for many institutions. Sharing portable facial devices among faculty members creates logistical delays when units are uncharged, misplaced, or undergoing maintenance.

2.5 Existing System Limitations

In accordance with Section 6 of the original project synopsis, the major limitations of existing attendance systems comprise:

- Manual attendance recording consumes valuable classroom time and reduces teaching efficiency.
- Traditional attendance methods are prone to human errors during attendance marking and record maintenance.
- Proxy attendance remains a significant challenge in manual and conventional attendance systems.
- Many institutions depend on dedicated biometric or facial recognition devices, increasing hardware dependency.
- Limited availability of attendance devices can lead to delays when multiple faculty members require the same device.
- Device maintenance, charging, and hardware failures may interrupt the attendance process.
- Existing systems often provide limited accessibility and cannot be easily accessed from different authorized devices.
- Generating attendance reports and maintaining records require additional administrative effort.
- Most attendance systems provide limited analytics and reporting capabilities for monitoring student attendance trends.
- Many systems lack flexible role-based access control for administrators, faculty members, and students.
- Existing systems provide limited flexibility for authorized faculty members to record attendance using secure web-based applications across multiple devices.

2.6 Proposed System Advantages

SAMS-FR addresses these limitations by decoupling facial recognition from dedicated hardware terminals. By processing camera video streams through standard web browsers, instructors can conduct attendance using existing personal laptops, tablets, or

smartphones. This architecture eliminates device procurement costs, prevents hardware sharing bottlenecks, and provides centralized data synchronization.

2.7 Summary

While manual methods consume teaching time and biometric hardware stations suffer from high costs and maintenance friction, web-based facial recognition provides an accessible, automated alternative. By embedding computer vision within a three-tier web application, SAMS-FR establishes an efficient, contactless attendance management workflow.

CHAPTER 3: REQUIREMENTS ANALYSIS

3.1 Functional Requirements

The functional requirements define the core operational capabilities implemented across SAMS-FR modules:

- **FR-01 (Authentication):** The system shall provide secure authentication for Administrators, Faculty Members, and Students using bcrypt password hashing and JSON Web Tokens (JWT).
- **FR-02 (User & Academic Management):** The system shall allow administrators to manage student, faculty, department, course offering, and batch records.
- **FR-03 (Biometric Enrollment):** The system shall allow administrators to enroll multi-angle biometric facial embeddings for registered students.
- **FR-04 (Attendance Sessions):** The system shall enable faculty members to initiate real-time attendance sessions for assigned classes using any camera-equipped browser.
- **FR-05 (Automated Face Recognition):** The system shall automatically detect student faces in video frames, extract 128D embeddings, match against enrolled profiles, and record attendance.
- **FR-06 (Manual Attendance Override):** The system shall allow faculty members to manually adjust attendance statuses (Present/Absent/Late/Excused) for exceptional cases.
- **FR-07 (Student Attendance Tracking):** The system shall enable students to view cumulative attendance percentages, subject-wise statistics, and attendance logs.
- **FR-08 (Reporting and Export Engine):** The system shall generate attendance reports and export them in PDF, Microsoft Excel (.xlsx), and CSV formats.

- **FR-09 (Security & Audit Logging):** The system shall record an immutable security audit log capturing user logins, biometric enrollments, and administrative updates.
- **FR-10 (Coursework & Notes Management):** The system shall allow faculty to publish assignments and study notes, and enable enrolled students to submit coursework and download materials.

3.2 Non-Functional Requirements

Non-functional requirements specify system quality attributes, performance constraints, and operational reliability:

- **NFR-01 (Performance):** Face detection and feature vector matching should complete within reasonable operational latency under standard classroom network conditions.
- **NFR-02 (Security):** API endpoints must be secured using cryptographically signed JWT cookies, role-based authorization guards, and CORS configuration.
- **NFR-03 (Scalability):** The database schema and service layer must support concurrent attendance sessions across departments without transaction deadlocks.
- **NFR-04 (Usability):** The user interface must be fully responsive across desktop workstations (1920x1080), laptops (1440x900), and mobile viewports.
- **NFR-05 (Reliability & Data Integrity):** The relational database must enforce foreign key integrity and transactional ACID consistency for all attendance records.

3.3 Admin Requirements

Administrative actors require comprehensive system governance tools: student directory management, faculty department allocations, course curriculum definitions, timetable scheduling, multi-view face profile enrollment, security audit log inspection, and global system configuration.

3.4 Faculty Requirements

Faculty members require class-level pedagogical tools: viewing assigned courses, launching live camera attendance sessions, reviewing student rosters, exporting attendance registers, uploading coursework assignments, and publishing lecture notes.

3.5 Student Requirements

Students require self-service capabilities: checking daily attendance status, performing face verification punch-ins, viewing subject-wise attendance analytics against the statutory 75% threshold, inspecting weekly timetables, and submitting assignments.

3.6 Hardware Requirements

The minimum and recommended hardware specifications for developing and executing SAMS-FR are presented in Table 3.2, directly matching Section 9 of the project synopsis:

Table 3.2: Hardware Requirements and Infrastructure Specifications

Hardware Component	Minimum Specification	Recommended Production Specification
Central Processor (CPU)	Intel Core i5 or equivalent	Intel Core i7 / AMD Ryzen 7 (10th Gen or higher)
System Memory (RAM)	Minimum 8 GB	16 GB DDR4/DDR5 Dual-Channel
Storage Drive	Minimum 256 GB SSD (or equivalent free space)	512 GB NVMe M.2 Solid-State Drive
Optical Camera Sensor	HD Webcam or Smartphone Camera	Full HD 1080p Webcam or Smartphone Camera
Internet Connection	Required for web access and database connectivity	High-Speed Campus LAN / Wi-Fi (50+ Mbps)

3.7 Software Requirements

The software requirements and library toolchain utilized in SAMS-FR are listed in Table 3.1, corresponding to Section 8 of the project synopsis:

Table 3.1: Software Requirements and Development Toolchain

Software	Purpose in Synopsis	Implemented Version / Package in SAMS-FR
Operating System	Windows 10/11 or Linux	Microsoft Windows 11 Pro 64-bit / Linux compatible
Programming Language	Python, JavaScript	Python 3.10.11 & JavaScript (ES2022 / React JSX)
Frontend	React.js, HTML5, CSS3, Bootstrap	React.js 19.2.8 with HTML5, CSS3, Vite 8.2 tooling
Backend	Flask	Flask 3.x with Flask-SQLAlchemy, Flask-JWT-Extended

Software	Purpose in Synopsis	Implemented Version / Package in SAMS-FR
Database	MySQL	MySQL Server 8.0 with PyMySQL database connector
Computer Vision Library	OpenCV	OpenCV (cv2) 4.x for real-time image pre-processing
Face Recognition Library	face_recognition	face_recognition (dlib 128D deep neural net)
Data Processing	Numpy, Pandas	NumPy 1.24+ (vector math) & Pandas 2.x (data export)
API Testing	Postman	Postman Desktop & Automated Pytest 9.1 test suite
Version Control	Git & GitHub	Git 2.4x on branch feature/frontend-auth-portals
Code Editor	Visual Studio Code	Visual Studio Code workspace with extensions
Web Browser	Google Chrome / Microsoft Edge	Google Chrome 120+ / Microsoft Edge 120+

3.8 Feasibility Analysis

A three-part feasibility assessment confirmed project viability:

- **Technical Feasibility:** Open-source libraries (OpenCV, dlib) integrate with standard browser camera APIs without requiring proprietary hardware.
- **Operational Feasibility:** Instructors initiate attendance with a single click, and students access self-service records through an intuitive user interface.
- **Economic Feasibility:** Utilizing existing institutional and personal devices eliminates dedicated biometric terminal acquisition and maintenance costs.

3.9 Development Methodology: Agile Model

In accordance with Section 10 of the synopsis, SAMS-FR followed the Agile Software Development Methodology. Development proceeded through iterative sprints, allowing continuous testing and enhancement of individual modules. Table 3.3 summarizes the Agile development phases:

Table 3.3: Agile Development Phases and Deliverables

Phase	Activities in Synopsis	Actual Deliverables in SAMS-FR
Requirement Analysis	Identify project requirements, objectives, and scope	Formal SRS, functional specifications, and user role matrix.
System Design	Prepare architecture, database design, UI design, and DFDs	Three-tier architecture, normalized ER model, DFD 0/1, and UI layouts.
Development	Develop frontend, backend, database, and AI modules	Flask REST APIs, React portal pages, dlib/OpenCV recognition engine.
Testing	Perform unit testing, integration testing, and system testing	Pytest suite (486 tests), static analysis (ESLint), and threshold tests.
Deployment	Deploy and demonstrate the application in a local environment	Verified local deployment on ports 5000/5173, connected to MySQL.
Maintenance	Fix bugs, improve performance, and add future enhancements	Optimized export engine, timetable integration, and security logging.

Table 3.4 outlines the core functional modules specified in Section 12 of the project synopsis:

Table 3.4: Functional Module Architecture and Responsibilities

Module Name	Description in Synopsis	Source Code Implementation
Authentication Module	Manages user login, logout, and secure role-based authentication	backend/app/routes/auth.py, frontend/src/pages/auth/Login.jsx
Admin Module	Enables administrator to manage students, faculty, departments, courses, records	backend/app/routes/admin.py, frontend/src/pages/admin/
Faculty Module	Allows faculty to initiate attendance sessions, manage classes, generate reports	backend/app/routes/faculty.py, frontend/src/pages/faculty/
Student Module	Enables students to view attendance history, percentage, and profile	backend/app/routes/student.py, frontend/src/pages/student/
AI Face Recognition Module	Performs face detection, face encoding, face recognition, validation	backend/app/services/cv/, backend/app/routes/recognition.py
Attendance Management Module	Automatically records attendance, maintains history, stores in database	backend/app/models/attendance_record.py, backend/app/routes/attendance.py
Reports & Analytics Module	Generates attendance reports, summaries, and statistical analysis	backend/app/routes/reports.py, backend/app/utils/export_engine/

Module Name	Description in Synopsis	Source Code Implementation
Data Management Module	Stores and manages student, faculty, course, and attendance data securely	backend/app/models/, MySQL database sams_db

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

SAMS-FR follows a three-tier architecture consisting of the presentation layer, application layer, and data layer, as specified in Section 11 of the project synopsis. The presentation layer is built with React.js, providing dedicated interfaces for administrators, faculty members, and students. The application layer is implemented in Flask, which handles authentication, attendance recording, report exports, and computer vision workflows through REST APIs. The data layer comprises the MySQL database and the AI Face Recognition Engine.

Figure 4.1 shows the system architecture diagram preserved from the original project synopsis:

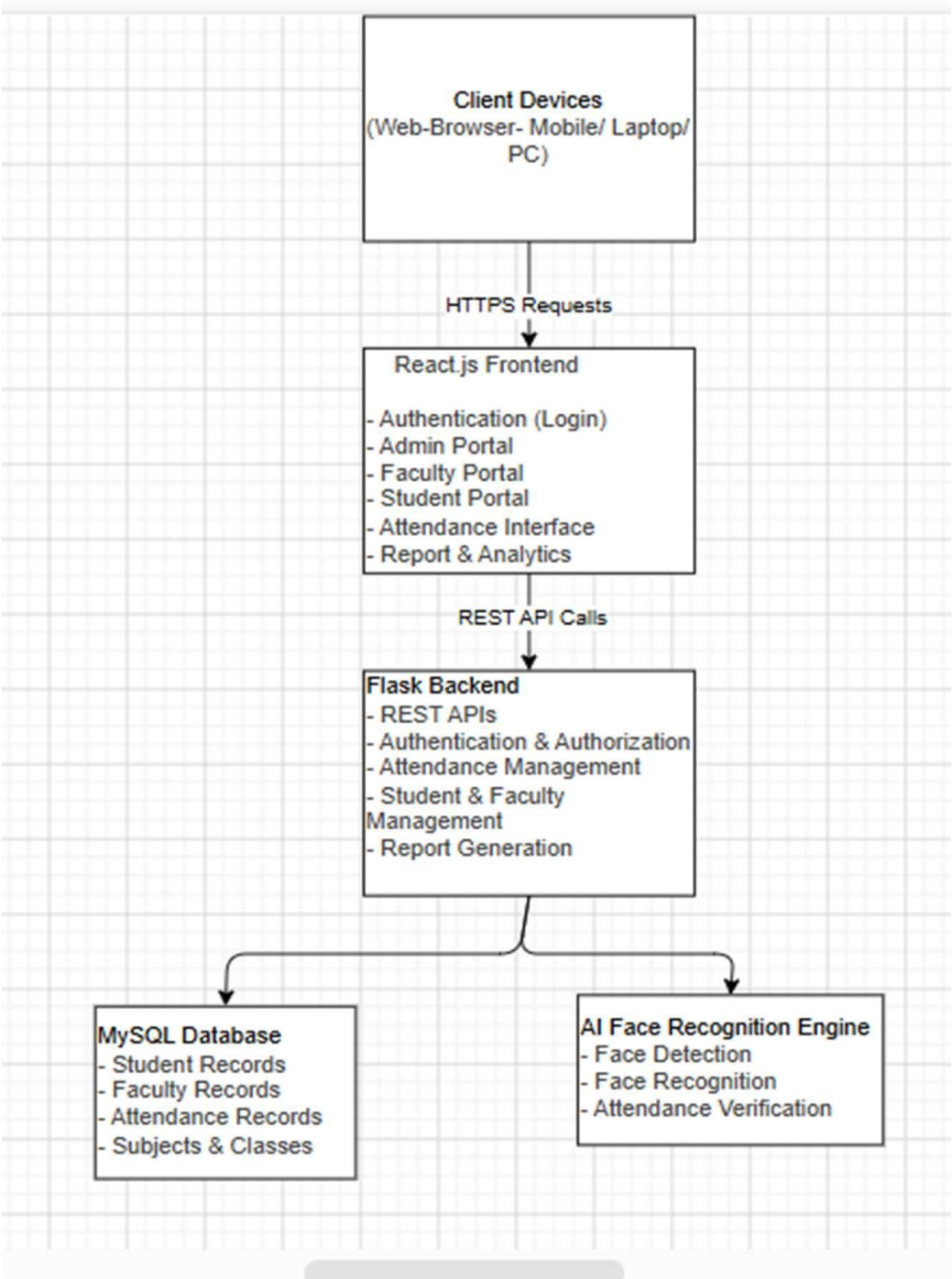


Figure 4.1: System Architecture of the Proposed AI-Powered Smart Attendance Management System (SAMS-FR)

4.2 System Modules

The functional modules operate cohesively across architectural tiers. Presentation components communicate with backend REST endpoints, which execute business logic, query relational models, and trigger facial recognition routines.

4.3 Data Flow Diagram Level 0

The Data Flow Diagram Level 0 (Context Diagram) represents external interactions between SAMS-FR and its primary actors: Administrator, Faculty Member, and Student. Administrators manage users and access statistics; faculty members authenticate and launch attendance sessions; students authenticate to inspect attendance logs. Figure 4.2 illustrates DFD Level 0 from the synopsis:

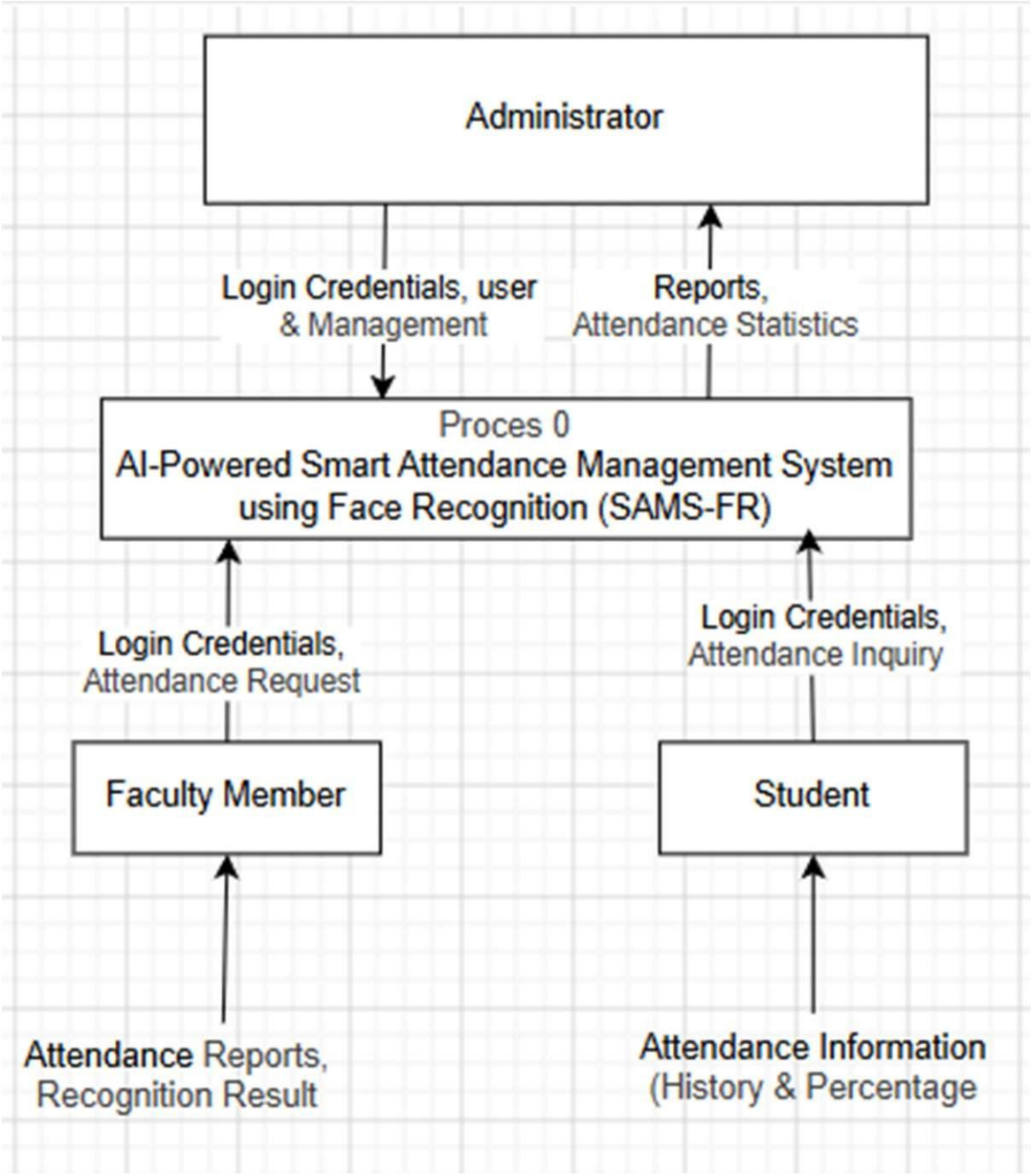


Figure 4.2: Data Flow Diagram (Level 0) — Context Diagram of SAMS-FR External Interactions

4.4 Data Flow Diagram Level 1

The Data Flow Diagram Level 1 decomposes the system into functional processes: 1.0 Authentication, 2.0 User & Academic Management, 3.0 Face Recognition & Attendance, 4.0 Attendance Management, and 5.0 Report Generation, depicting data exchange with persistent data stores. Figure 4.3 illustrates DFD Level 1 from the synopsis:


Administrator
Faculty Member
Student
Login Credentials
1.0 Authentication
Authenticated User
2.0 User & Academic Management
D1 Student Database
D2 Faculty Database
D3 Subject Database
D6 Department/ Program/ Section DB
Faculty Member
Start Attendance
3.0 Face Recognition & Attendance
Read Subject
Read Face Data
D3 Subject Database
D4 Face Data
Recognized Result
4.0 Attendance Management
Store Attendance
D5 Attendance Database
Attendance Records
5.0 Report Generation
Attendance Reports
Attendance Reports
Attendance Information (History & Percentage)
Administrator
Faculty Member
Student

Figure 4.3: Data Flow Diagram (Level 1) — Functional Process Flow and Data Stores

4.5 Use Case Diagram

The Use Case Diagram defines functional interactions between the system boundary and the three primary user roles: Administrator, Faculty Member, and Student. Figure 4.4 displays the Use Case model from the project synopsis:



Figure 4.4: Use Case Diagram of SAMS-FR Defining Administrator, Faculty, and Student Roles

4.6 Entity Relationship Diagram

The database structure follows a normalized Entity Relationship (ER) model. Major entities include Administrator, Department, Program, Semester, Section, Student, Faculty, Subject, Faculty Assignment, Face Data, Attendance Session, and Attendance. Figure 4.5 shows the ER Diagram from the project synopsis:

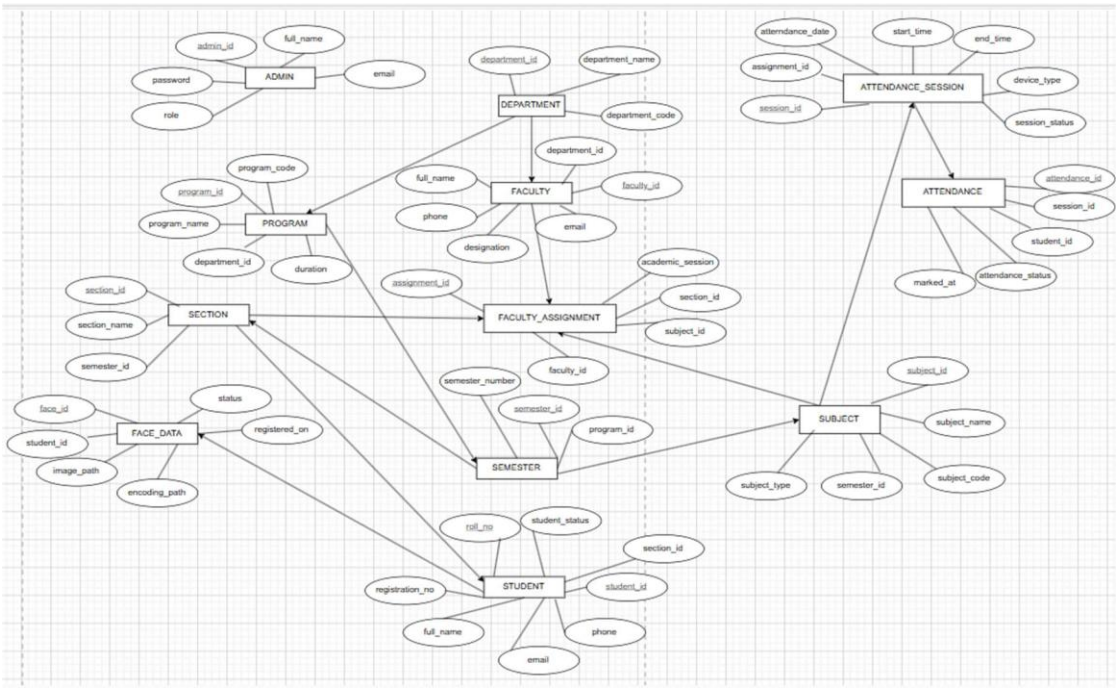


Figure 4.5: Entity Relationship (ER) Diagram of the Proposed SAMS-FR Relational Database

4.7 Database Design

Table 4.1 maps the conceptual ER entities from the synopsis to their physical MySQL table implementations:

Table 4.1: Core Relational Database Schema Entities, Keys, and Cardinalities

Synopsis Entity	Physical Table	Primary Key	Foreign Keys	Relationships & Purpose
Administrator / User	users	user_id	None	1:1 with Student or Faculty; stores email, role, and password hash.
Department	departments	department_id	None	1:N with Faculty, Students, and Courses; models academic departments.
Student	students	student_id	user_id, department_id	1:1 with User, 1:N with Enrollments, FaceProfiles, AttendanceRecords.

Synopsis Entity	Physical Table	Primary Key	Foreign Keys	Relationships & Purpose
Faculty	faculty	faculty_id	user_id, department_id	1:1 with User, 1:N with CourseOfferings and AttendanceSessions.
Subject / Course	courses	course_id	department_id	1:N with CourseOfferings; defines academic subjects and credits.
Faculty Assignment	course_offerings	offering_id	course_id, faculty_id	Maps courses, instructors, sections, and semesters.
Enrollment	enrollments	enrollment_id	student_id, offering_id	Associates students with active offerings to verify attendance eligibility.
Face Data	face_profiles	face_profile_id	student_id	1:N with Student; stores serialized 128D facial embeddings.
Attendance Session	attendance_sessions	session_id	offering_id, faculty_id	Represents an individual class period and parent of records.
Attendance	attendance_records	record_id	session_id, student_id	Stores atomic status (PRESENT/ABSENT/LATE), method, and timestamp.
Timetable	timetable_entries	entry_id	offering_id, faculty_id	Schedules day of week, timeslot, and room allocation.
Activity Log	activity_logs	log_id	actor_id (User)	Immutable audit trail recording administrative and user actions.

4.8 Face Recognition Workflow

The facial recognition sub-system executes a structured pipeline: (1) Frame acquisition from the browser client; (2) Face detection isolating bounding boxes; (3) 68-point facial landmark alignment; (4) Deep metric feature extraction producing a 128-dimensional embedding vector; (5) Metric comparison against enrolled student profiles using calibrated thresholds.

4.9 Attendance Workflow

During a live class session, the faculty member launches the attendance session interface. Incoming video frames are processed to detect faces. When a student's facial embedding matches an enrolled profile with sufficient similarity, the system marks the student present, updating the session roster and preventing duplicate records.

4.10 Security Design

Security architecture incorporates role-based authorization guards, bcrypt password hashing with work factor 12, and JWT tokens stored in secure HTTP-only cookies. An in-memory blocklist (`token_blocklist.py`) supports instant token revocation upon logout.

CHAPTER 5: SYSTEM IMPLEMENTATION

5.1 Technology Stack

The concrete implementation of SAMS-FR utilizes Python 3.10 and Flask on the backend, React 19 and Vite on the frontend, and MySQL 8.0 for relational storage, supported by OpenCV and dlib for computer vision.

5.2 Frontend Implementation

The React Single-Page Application is organized into modular directories: `src/pages/` (role-specific portal views), `src/components/` (reusable UI controls such as `DataTable.jsx` and `LiveFaceCapture.jsx`), `src/context/` (`AuthContext.jsx`), and `src/services/` (Axios API services). Code splitting via `React.lazy()` ensures fast initial page loads.

5.3 Backend Implementation

The Flask application uses the application factory pattern (`create_app()` in `backend/app/__init__.py`). Endpoints are grouped into modular blueprints: `auth_bp`, `admin_bp`, `faculty_bp`, `student_bp`, `recognition_bp`, and `reports_bp`. Flask-JWT-Extended manages token issuance and claims verification.

5.4 Database Implementation

Database tables are mapped using SQLAlchemy 2.0 ORM. Schema migrations are managed via Flask-Migrate and Alembic. Indexed columns on foreign keys and date fields optimize query performance.

5.5 Authentication and Authorization

User credentials are authenticated by comparing submitted passwords against bcrypt hashes. Successful login issues an HTTP-only JWT cookie containing user identity and role

claims (`ADMIN`, `FACULTY`, `STUDENT`). Endpoint decorators (`@admin_required`, `@jwt_required()`) enforce access boundaries.

5.6 Face Recognition Implementation

The computer vision engine in `backend/app/services/cv/` implements `FaceService` and `RecognitionService`. Faces are detected, aligned, and converted into 128D NumPy vector embeddings. Vector matching evaluates cosine similarity and Euclidean distance against enrolled profiles.

5.7 Attendance Management

Attendance session controllers in `attendance.py` manage session lifecycles. Real-time face punches verify student enrollment in the specific course offering prior to marking attendance, preventing unauthorized or out-of-class records.

5.8 Academic Management

Academic management services coordinate departments, courses, offerings, batch timelines, and the master timetable matrix. Temporary faculty adjustments allow instructors to cover classes without modifying core course assignments.

5.9 Assignment and Study Material Module

Coursework modules enable faculty to create assignments, upload worksheets, set deadlines, and configure allowed submission formats (file upload, text, or both). Students submit work through their portal, and faculty record numerical scores and qualitative feedback.

5.10 Reporting and Export Engine

The export engine (`backend/app/utils/export_engine/`) generates formatted PDF documents via ReportLab, Excel spreadsheets via OpenPyXL, and CSV files, enabling flexible administrative data export.

5.11 API Integration

Communication between React and Flask relies on RESTful JSON APIs. The frontend Axios client uses request interceptors for credentials and response interceptors to handle session expirations.

CHAPTER 6: APPLICATION DEMONSTRATION

This chapter demonstrates the operational interfaces of SAMS-FR across its three portals. All figures displayed herein represent genuine desktop screenshots captured from the running application (1440x900 resolution) connected to the active MySQL database. Explanatory text introduces each functional component beside its corresponding visual evidence.

6.1 Authentication Portal

The Authentication Portal provides a centralized security gateway (`/auth`). It features a multi-role selector allowing users to choose their appropriate operational portal (Administrator, Faculty, or Student). Selecting a role dynamically updates form labels, placeholder text, and visual accents to guide the user. Figure 6.1 demonstrates the central login screen:

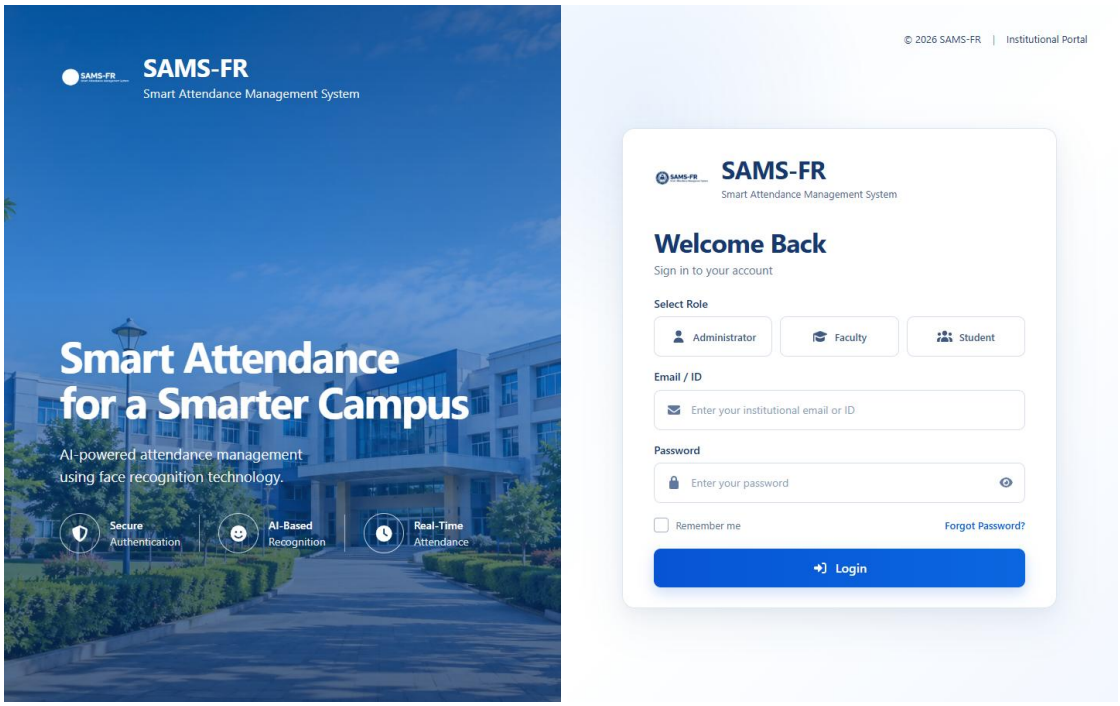


Figure 6.1: SAMS-FR Central Multi-Role Authentication Gateway

When the user selects the Administrator role, the portal transitions to administrative mode. The form prompts for authorized administrative email credentials and password, verifying privileges prior to granting dashboard access. Figure 6.2 illustrates the Administrator role selection view:

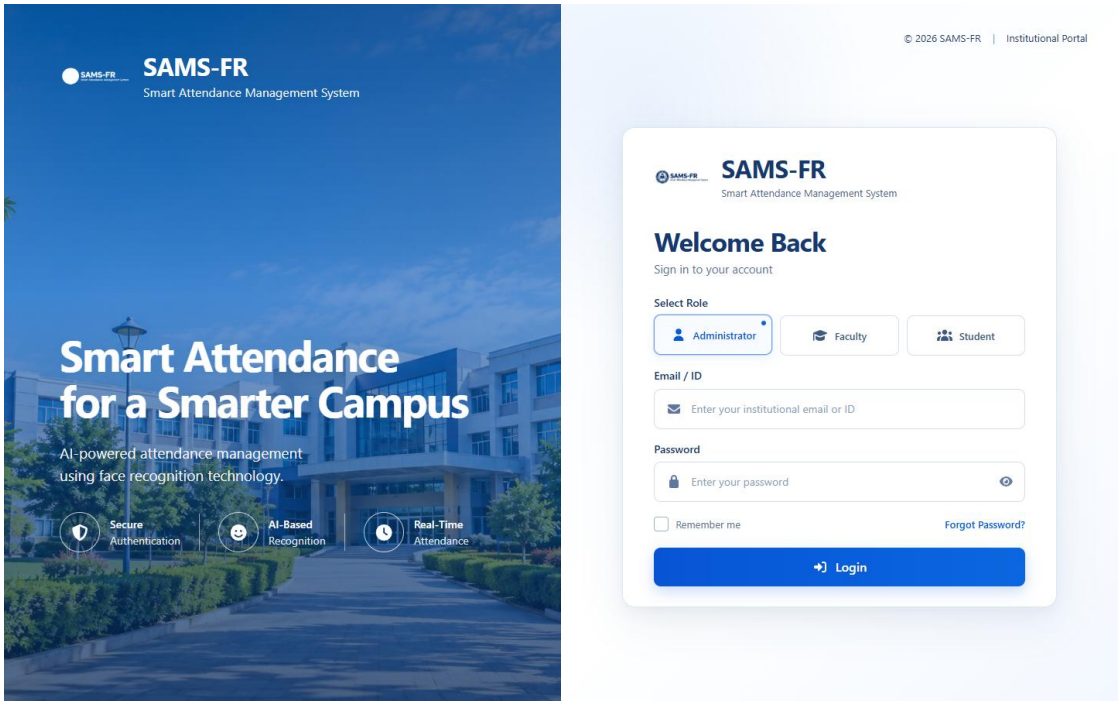


Figure 6.2: Administrator Role Selection and Authentication View

6.2 Administrator Portal

Upon successful authentication, administrators access the Administrator Dashboard (`/admin/dashboard`). The dashboard provides high-level institutional metrics, including total enrolled students, active faculty counts, registered course offerings, and daily attendance averages. Quick action cards allow rapid navigation to key management workflows. Figure 6.3 displays the central administrator dashboard:

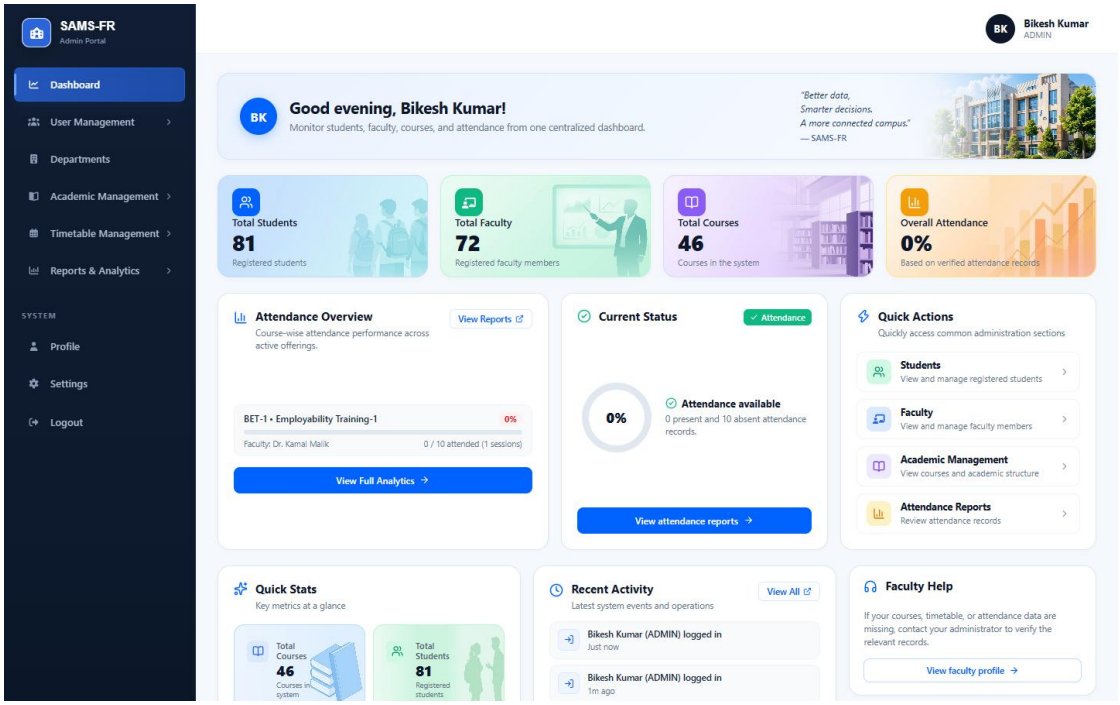


Figure 6.3: Administrator Dashboard and Real-Time Institutional Analytics

Student lifecycle management is handled via the Student Directory (`/admin/students`). The interface features real-time search, semester and section filtering, enrollment number tracking, and profile status badges. Administrators can inspect student details, initiate biometric enrollments, or update enrollment records. Figure 6.4 shows the student directory interface:

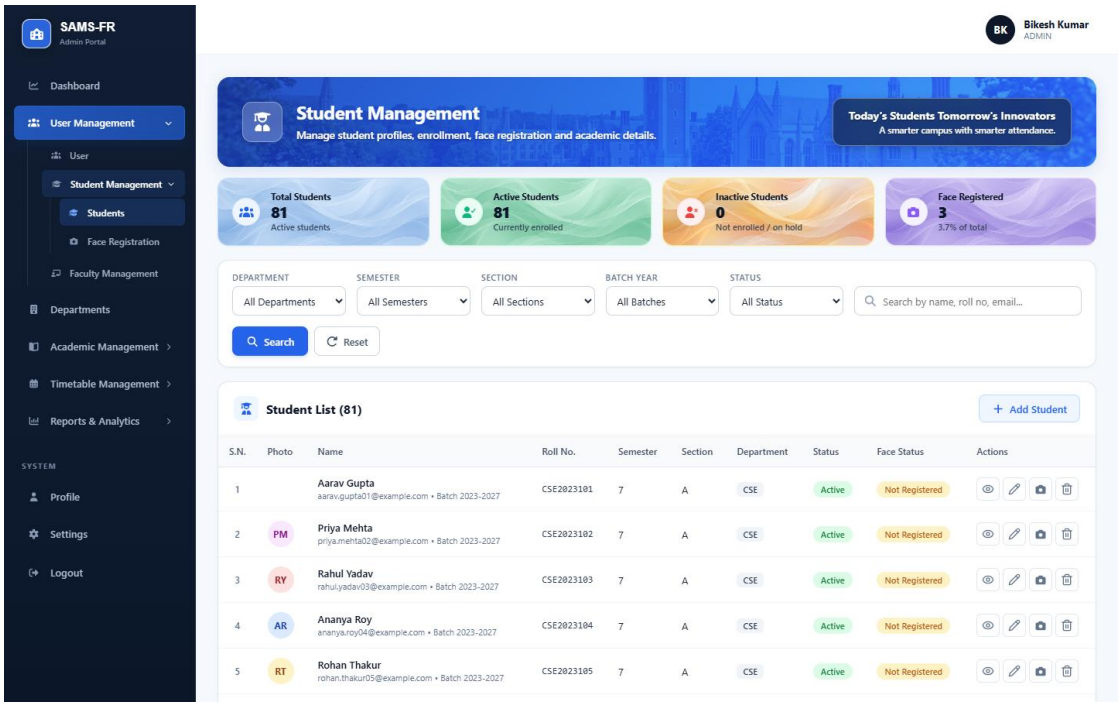


Figure 6.4: Central Student Directory and Cohort Filter Management

Biometric enrollment is conducted through the Face Registration interface (`/admin/face-registration`). The system activates the workstation camera to capture facial images from front, left, and right angles. Automated checks verify face detection and illumination before computing 128D embeddings and saving the profile. Figure 6.5 illustrates the face registration interface:

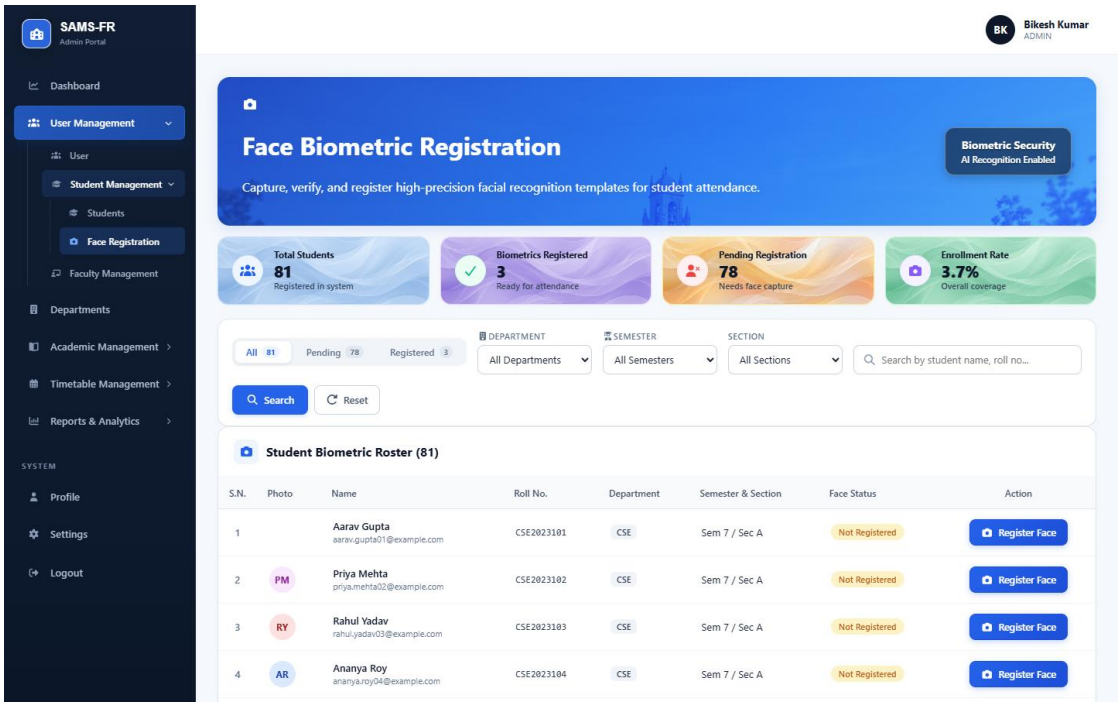


Figure 6.5: Multi-Angle Biometric Face Registration Interface

Faculty oversight is centralized in the Faculty Directory (`/admin/faculty`). Administrators manage instructor profiles, department assignments, employee identification numbers, and contact details. Figure 6.6 demonstrates the faculty management directory:

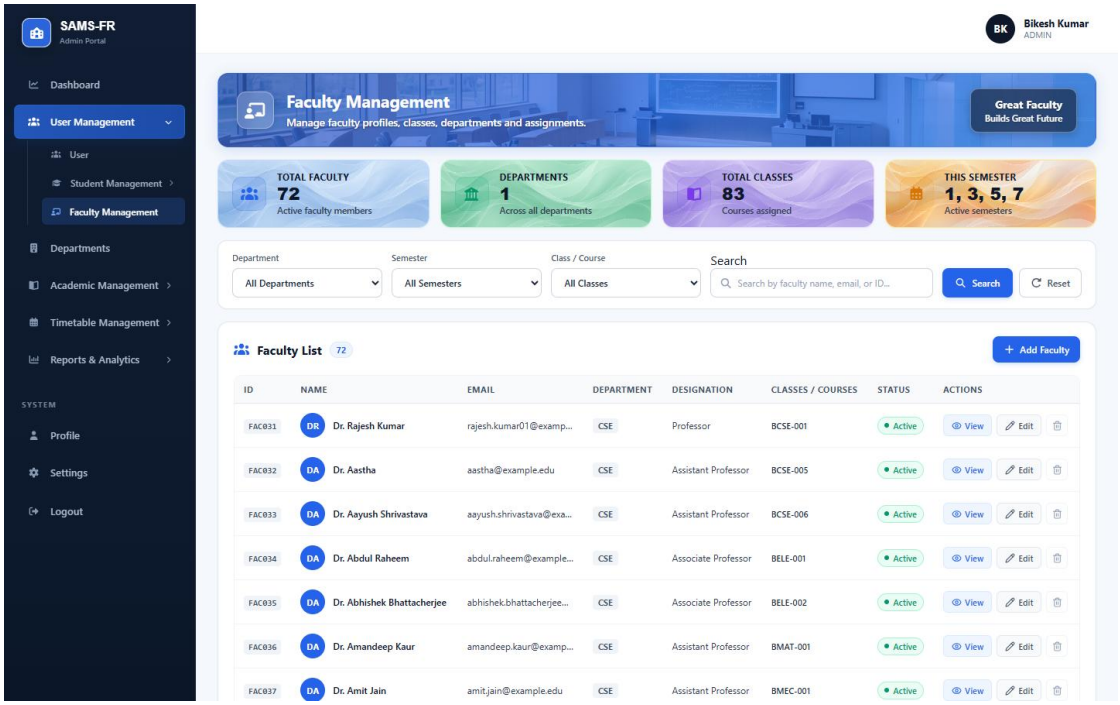


Figure 6.6: Faculty Directory and Departmental Allocation

Institutional departments are structured through the Department Management interface (`/admin/departments`). Administrators manage department codes, names, faculty head allocations, and active course totals. Figure 6.7 displays the department management view:

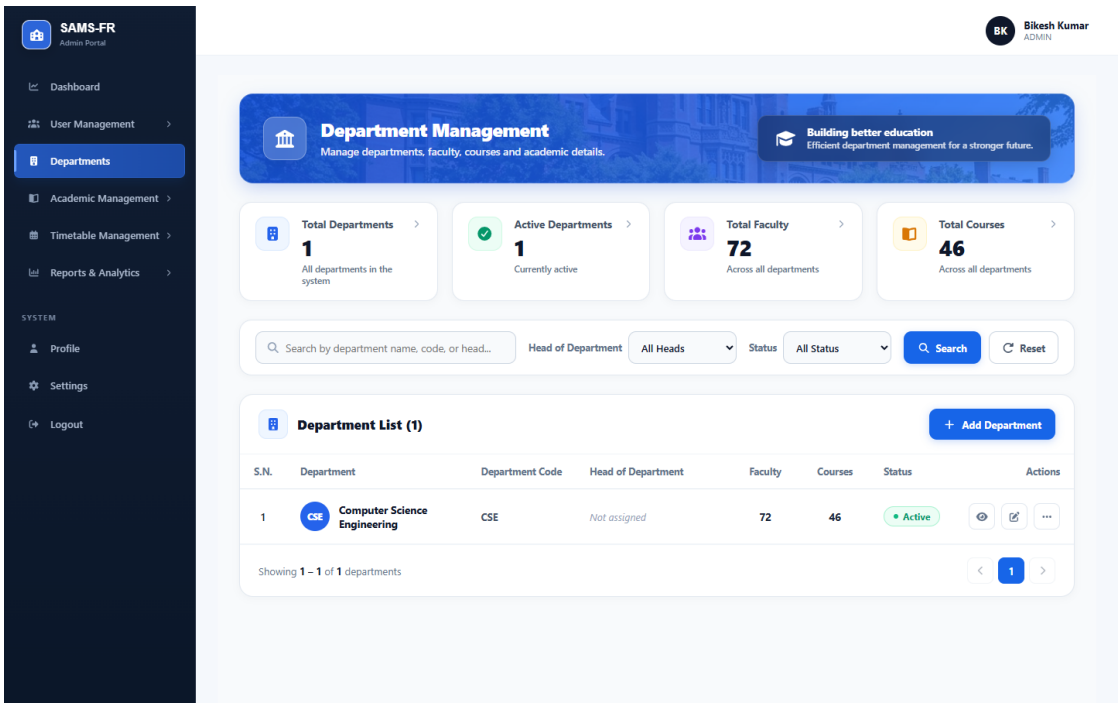


Figure 6.7: Academic Department Structure Management

Curriculum offerings are coordinated in Academic Management (`/admin/academic-management`). This interface maps course codes, academic credits, semester terms, and section allocations to assigned faculty instructors. Figure 6.8 shows the academic course offerings management interface:

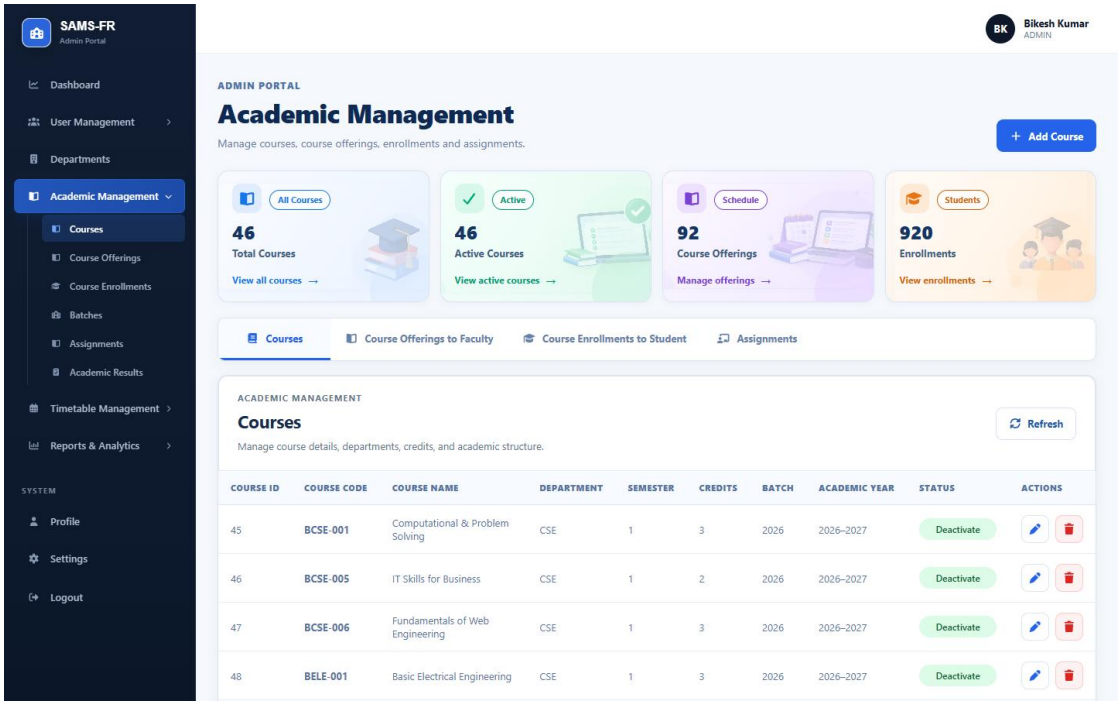


Figure 6.8: Academic Curriculum and Course Offering Allocation

Institutional examination moderation is administered via Academic Results (`/admin/academic-results`). Administrators review marks submitted by faculty members, verify grading compliance, and officially approve and publish results. Figure 6.9 shows the academic results moderation hub:

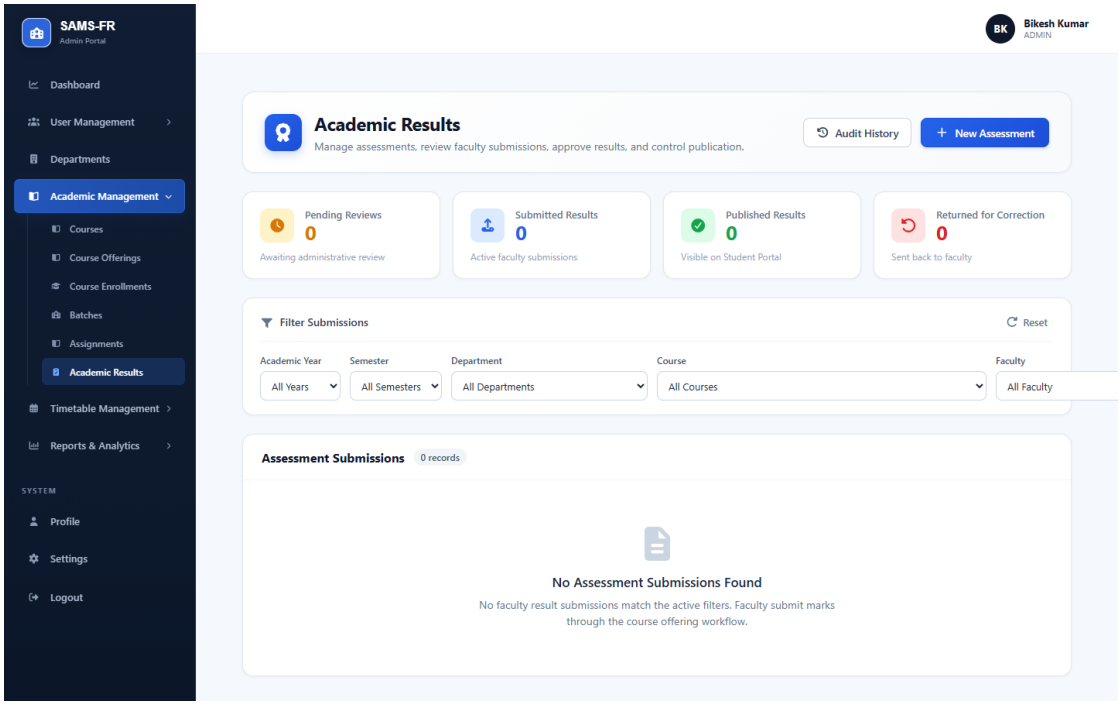


Figure 6.9: Institutional Academic Result Moderation and Publishing

Master class timetables are scheduled via Timetable Management (`/admin/timetable`). The weekly matrix displays scheduled periods, assigned instructors, and room numbers, while preventing scheduling conflicts. Figure 6.10 shows the master timetable scheduling matrix:

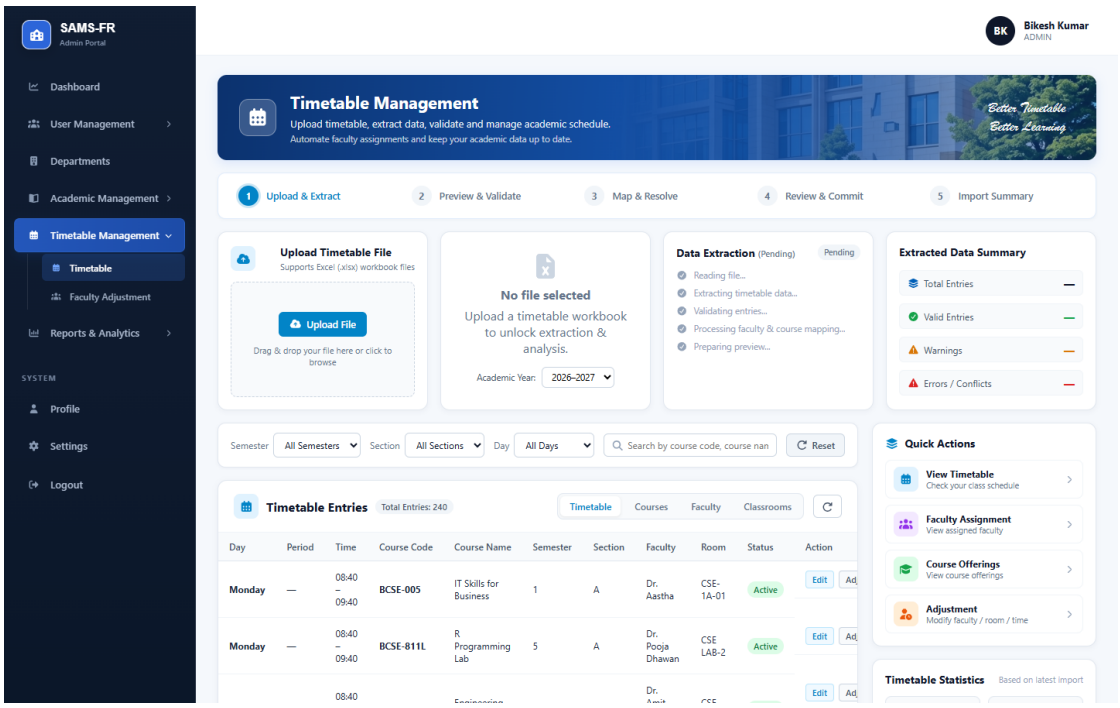


Figure 6.10: Master Timetable Matrix and Room Scheduling

Institutional attendance analytics are aggregated in Attendance Reports (`/admin/attendance-reports`). Administrators inspect attendance trends across departments, identify low-attendance cohorts, and export official reports in PDF, Excel, or CSV formats. Figure 6.11 illustrates the attendance reports interface:

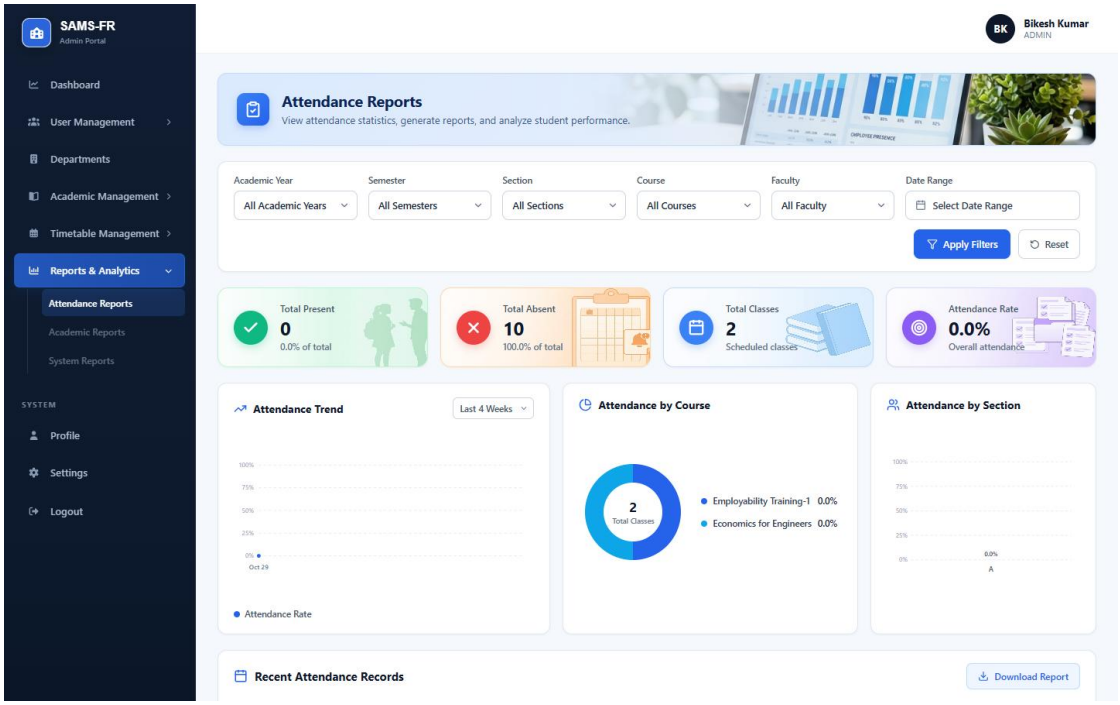


Figure 6.11: Institutional Attendance Analytics and Export Engine

System security and administrative auditability are maintained through Activity Logs (`/admin/activity-logs`). The immutable event stream records user logins, face profile enrollments, timetable edits, and attendance transactions. Figure 6.12 illustrates the activity log viewer:

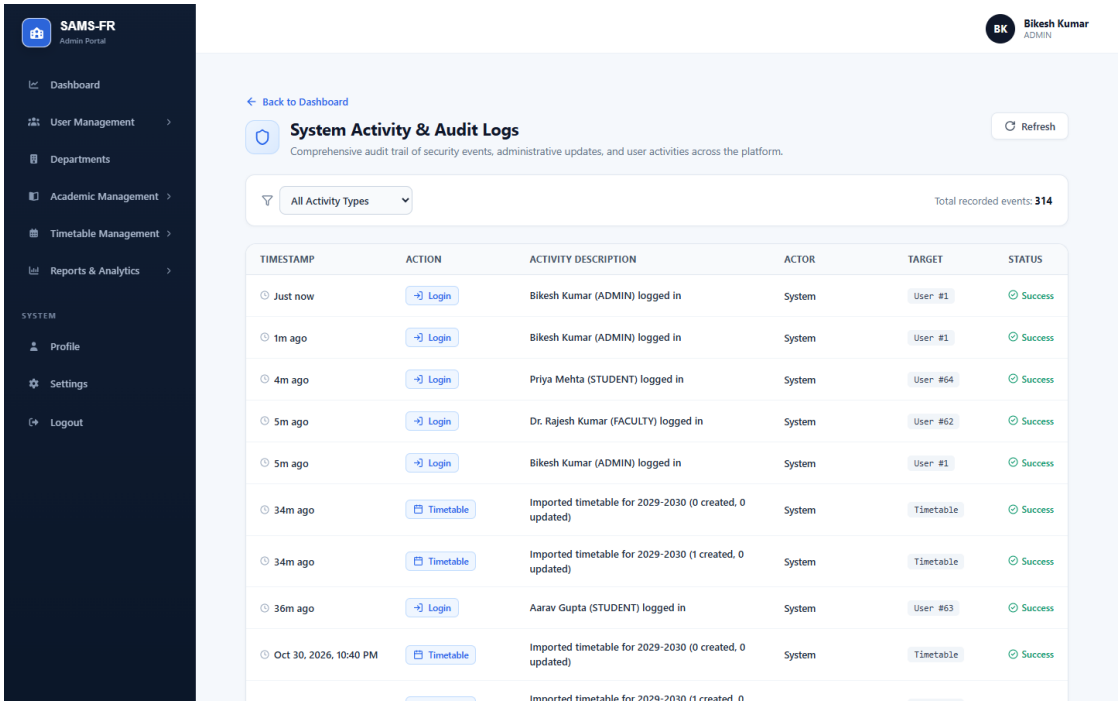


Figure 6.12: Security and Audit Activity Event Log

6.3 Faculty Portal

Faculty members authenticate into the Faculty Dashboard (`/faculty/dashboard`). The dashboard presents immediate operational information: today's scheduled classes, assigned course offerings, pending coursework submissions, and average class attendance. Figure 6.13 displays the faculty dashboard:

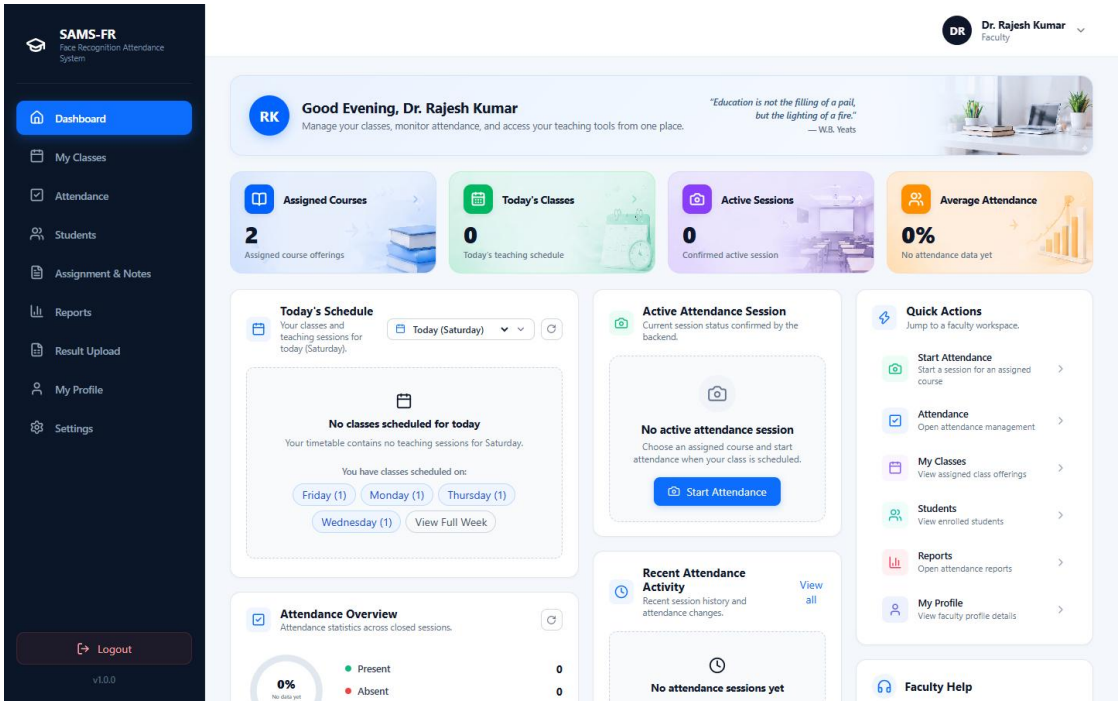


Figure 6.13: Faculty Portal Overview and Today's Class Schedule

Assigned courses and class rosters are organized under My Classes (`/faculty/classes`). Instructors inspect semester offerings, enrolled student counts, and export student rosters in PDF, Excel, or CSV formats. Figure 6.14 displays the assigned classes interface:

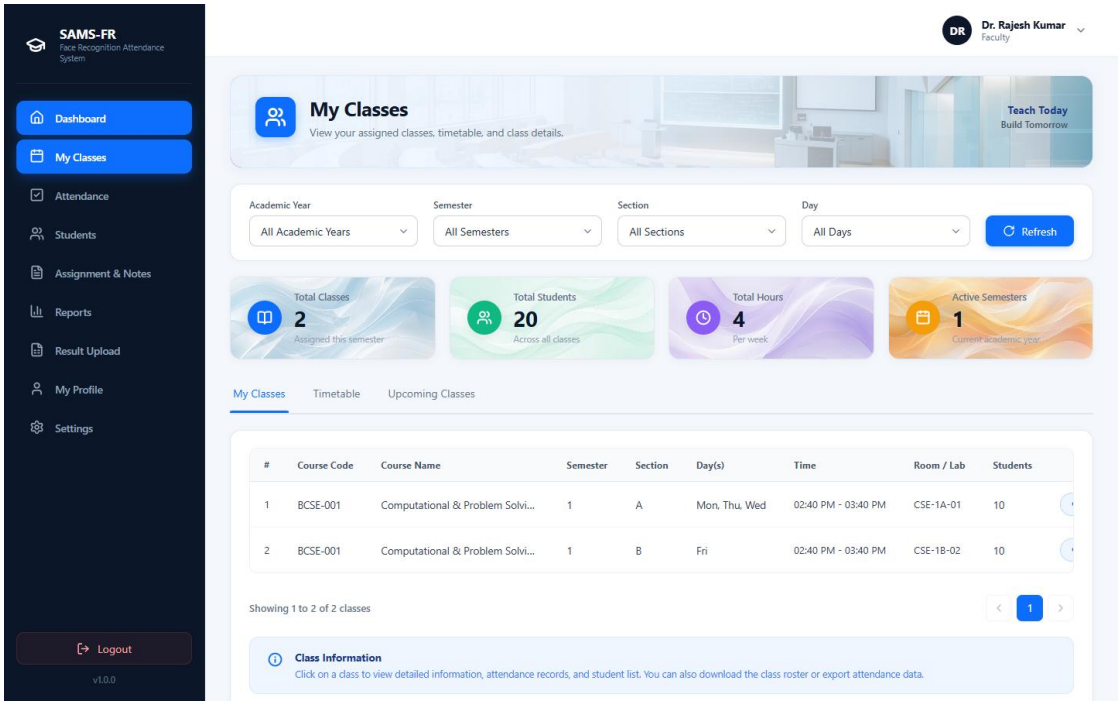


Figure 6.14: Assigned Course Offerings and Class Roster Directory

Attendance execution takes place in the Attendance Session interface (`/faculty/attendance`). The instructor selects an assigned class and starts the session. The system activates the device camera, streams frames to the recognition service, and identifies enrolled students in real time with visual status feedback. Figure 6.15 demonstrates the faculty attendance session control interface:

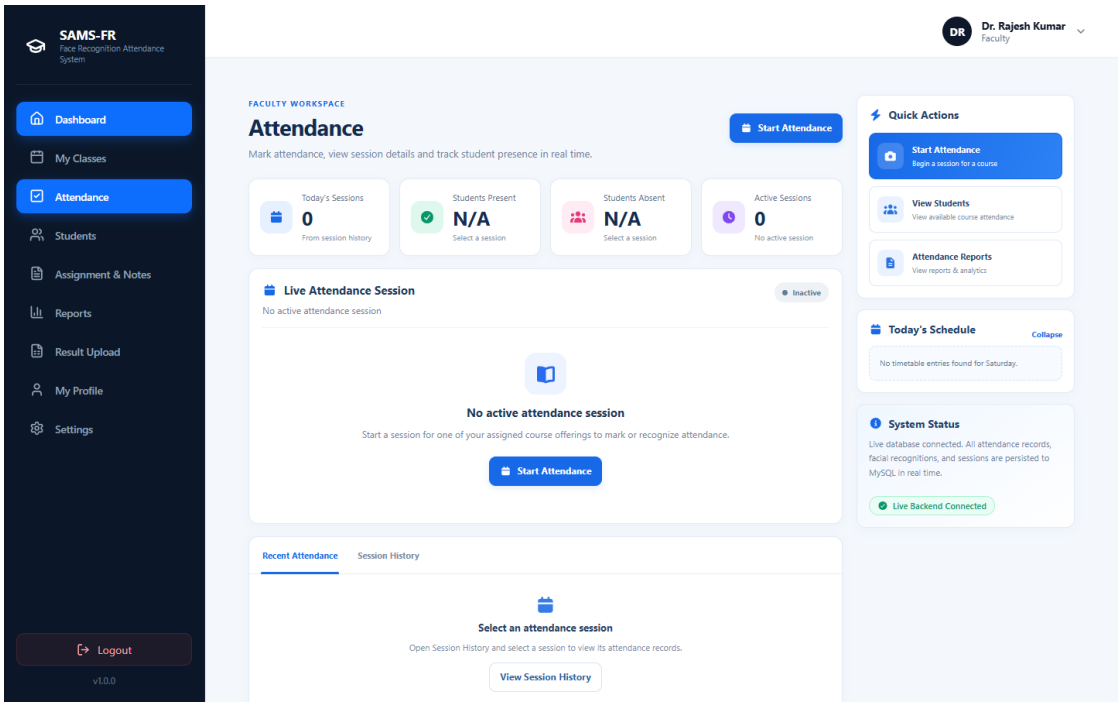


Figure 6.15: Real-Time Computer Vision Attendance Session Control

Course-level attendance registers are reviewed under Faculty Reports (`/faculty/reports`). Instructors inspect session logs, filter records by date range, and export attendance registers. Figure 6.16 demonstrates the faculty course attendance report view:

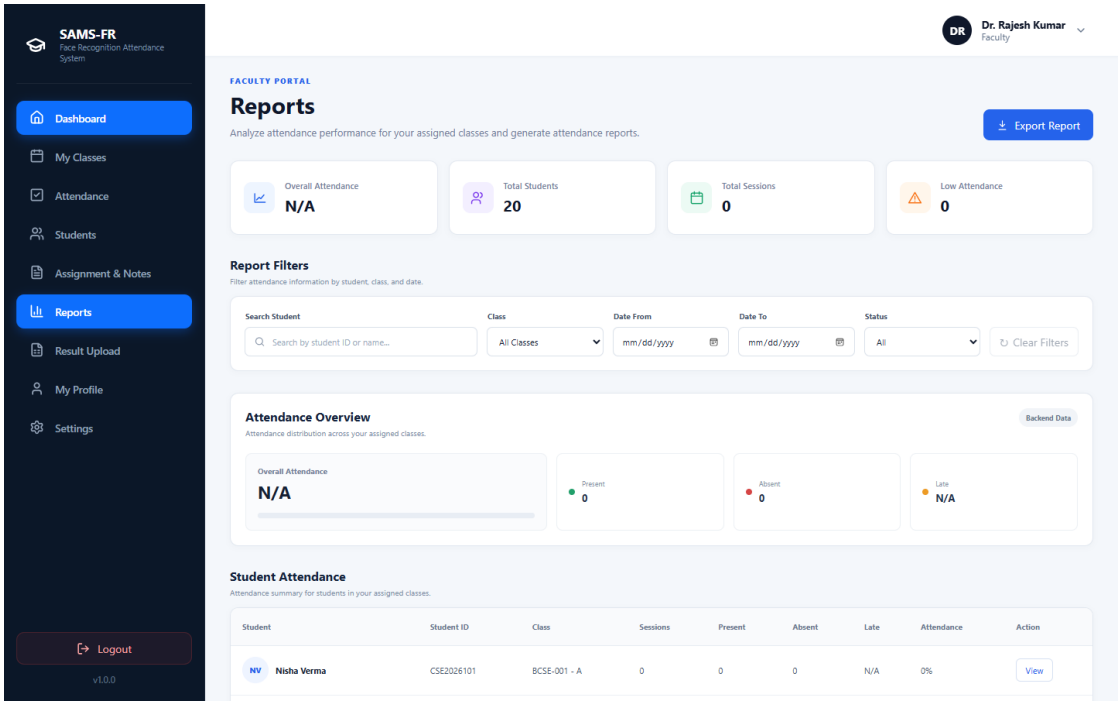


Figure 6.16: Course Attendance Register and Export Interface

Coursework distribution is managed in Assignments & Notes (`/faculty/assignments`). Faculty create assignments, set deadlines, upload lecture notes, and track student submissions. Figure 6.17 displays the coursework management hub:

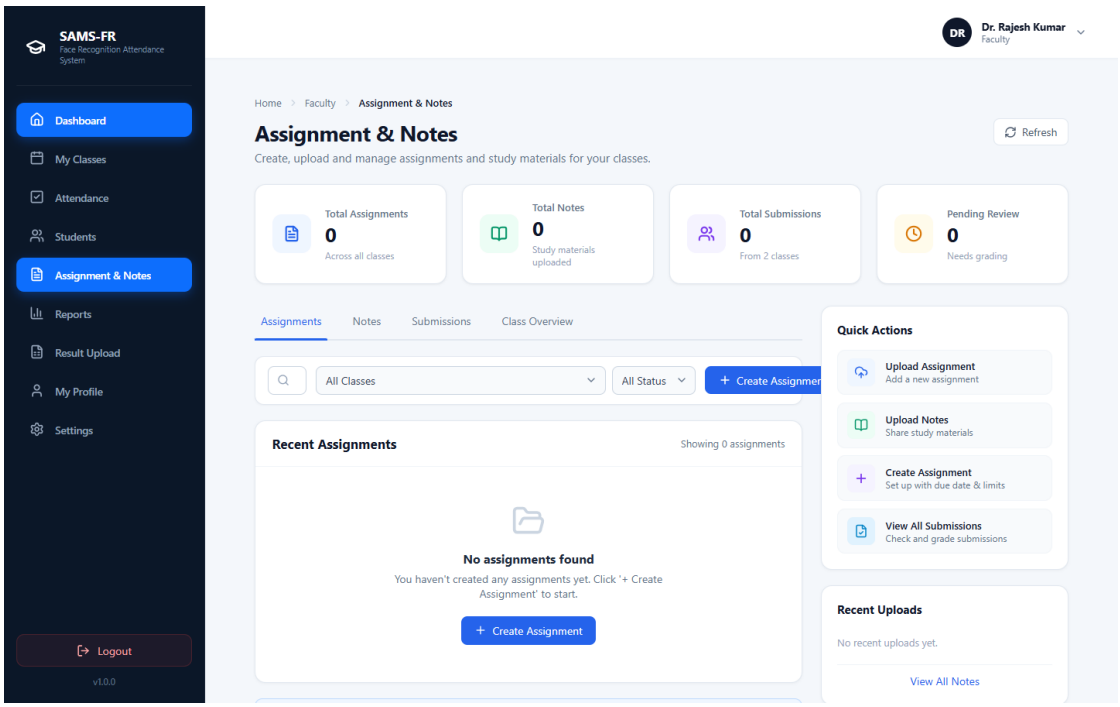


Figure 6.17: Coursework Assignment and Study Notes Hub

Academic grading is performed in Result Upload (`/faculty/results`). Faculty enter internal assessment scores and submit grade sheets for administrative moderation. Figure 6.18 shows the marks entry interface:

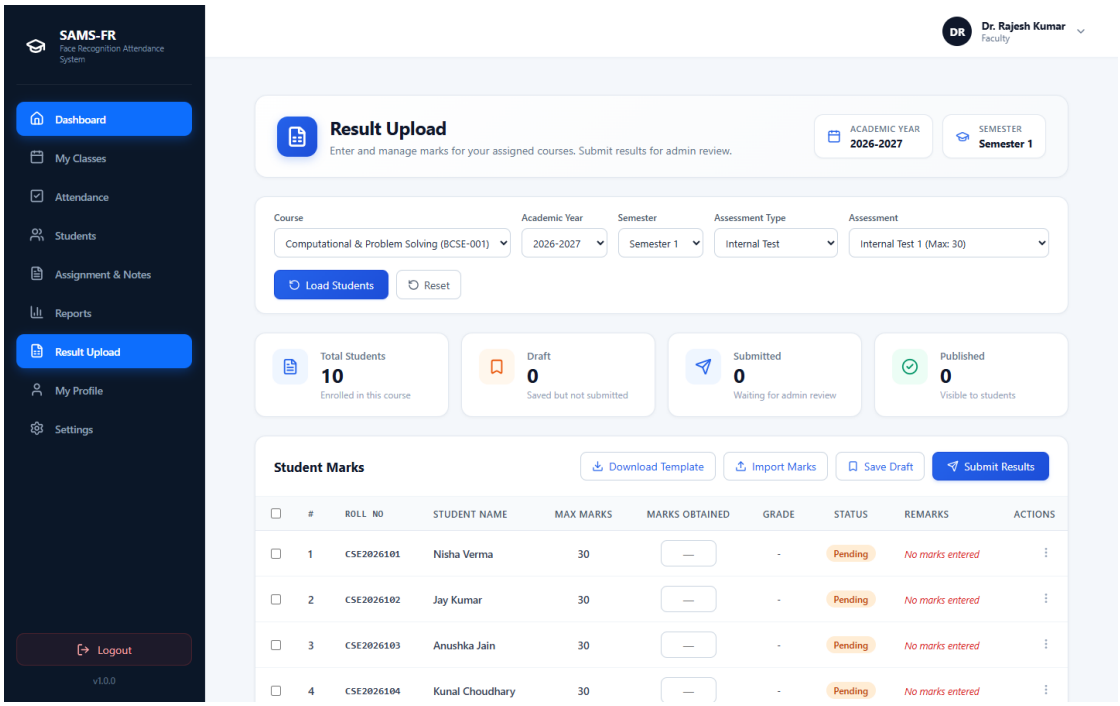


Figure 6.18: Internal Assessment Marks Entry and Evaluation

6.4 Student Portal

Students authenticate into the Student Dashboard (`/student/dashboard`). The dashboard displays cumulative attendance percentage, alert banners regarding the statutory 75% threshold, today's schedule, and pending coursework deadlines. Figure 6.19 displays the student dashboard:

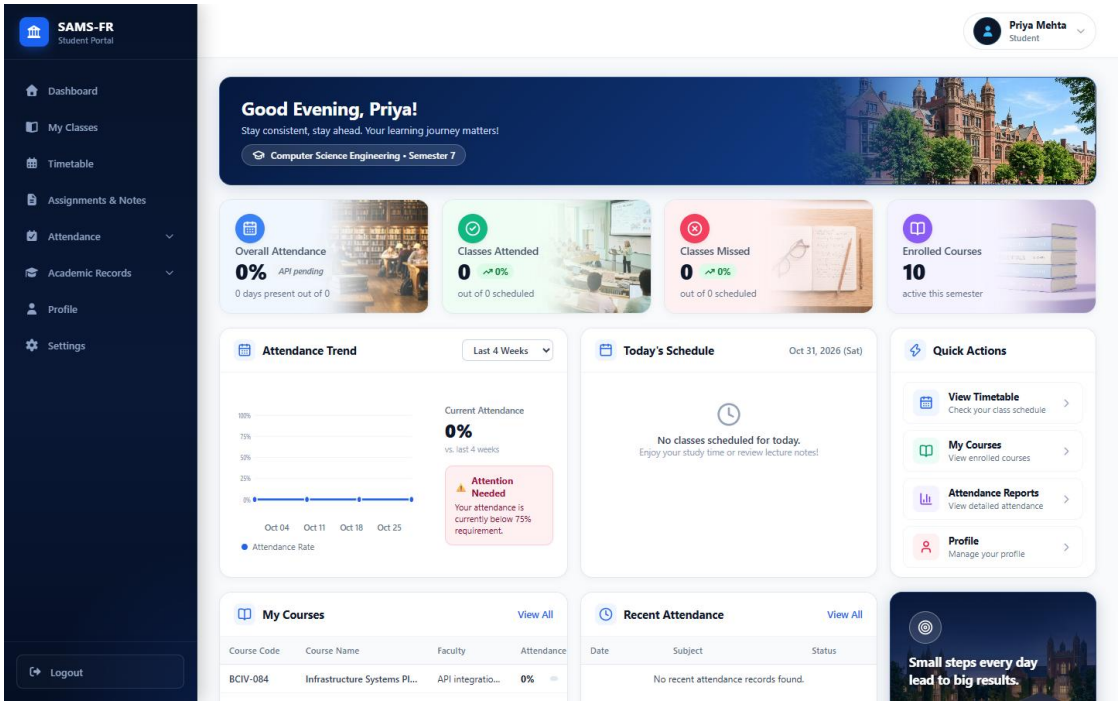


Figure 6.19: Student Portal Overview and Aggregate Attendance Metric

Students perform contactless self-service attendance verification through Daily Punch (`/student/attendance`). The camera feed detects the student's face, compares embeddings against their profile, and confirms attendance. Figure 6.20 displays the student daily punch interface:

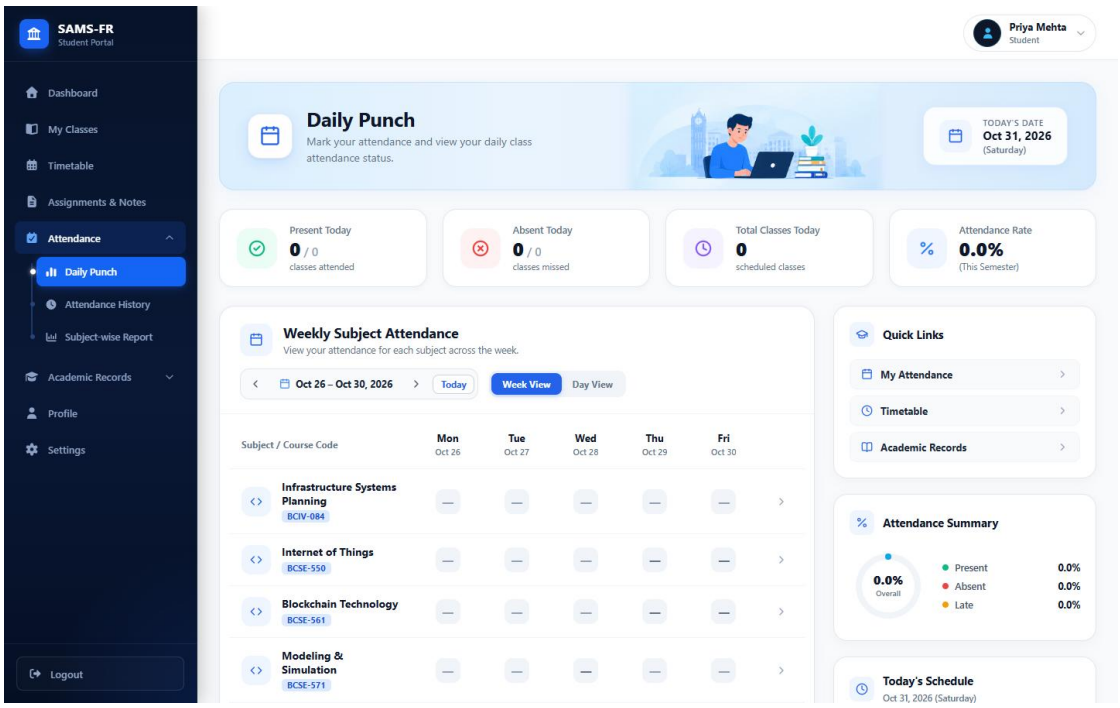


Figure 6.20: Contactless Student Daily Punch Face Verification Interface

A chronological record of past attendance events is available in Attendance History (`/student/attendance-history`). Students inspect dates, course names, and marked statuses. Figure 6.21 displays the attendance history interface:

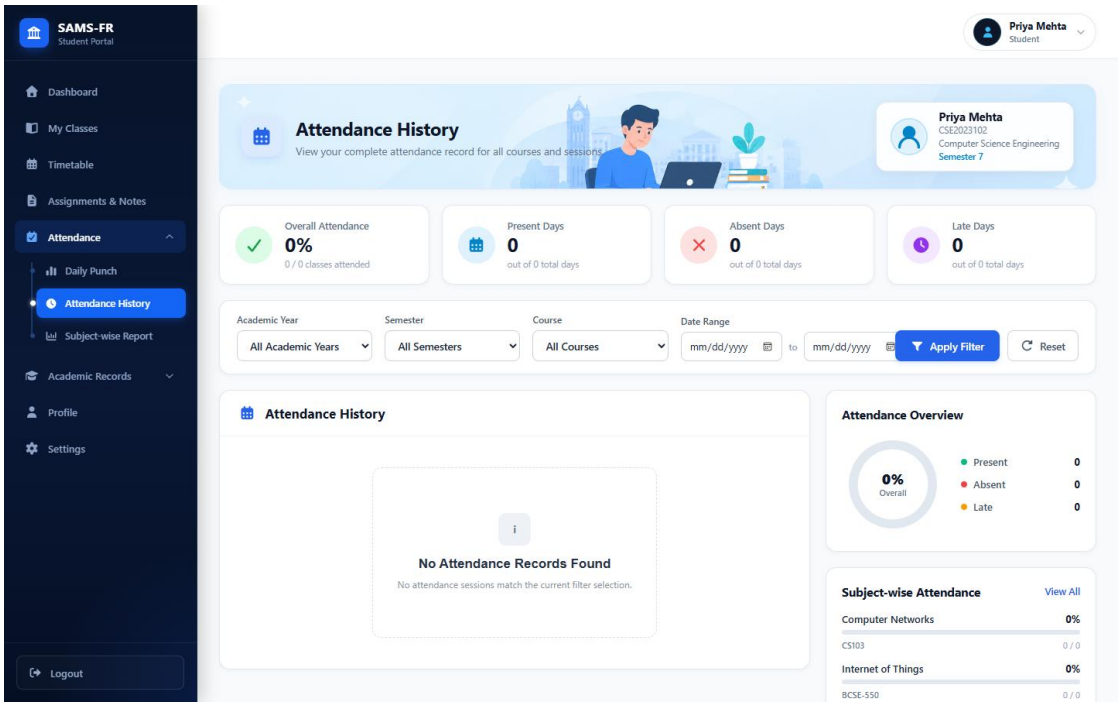


Figure 6.21: Chronological Student Attendance History Log

Enrolled courses are displayed in My Classes (`/student/courses`). Students review enrolled subjects, instructor details, credits, and syllabus progression. Figure 6.22 displays the student course enrollment directory:

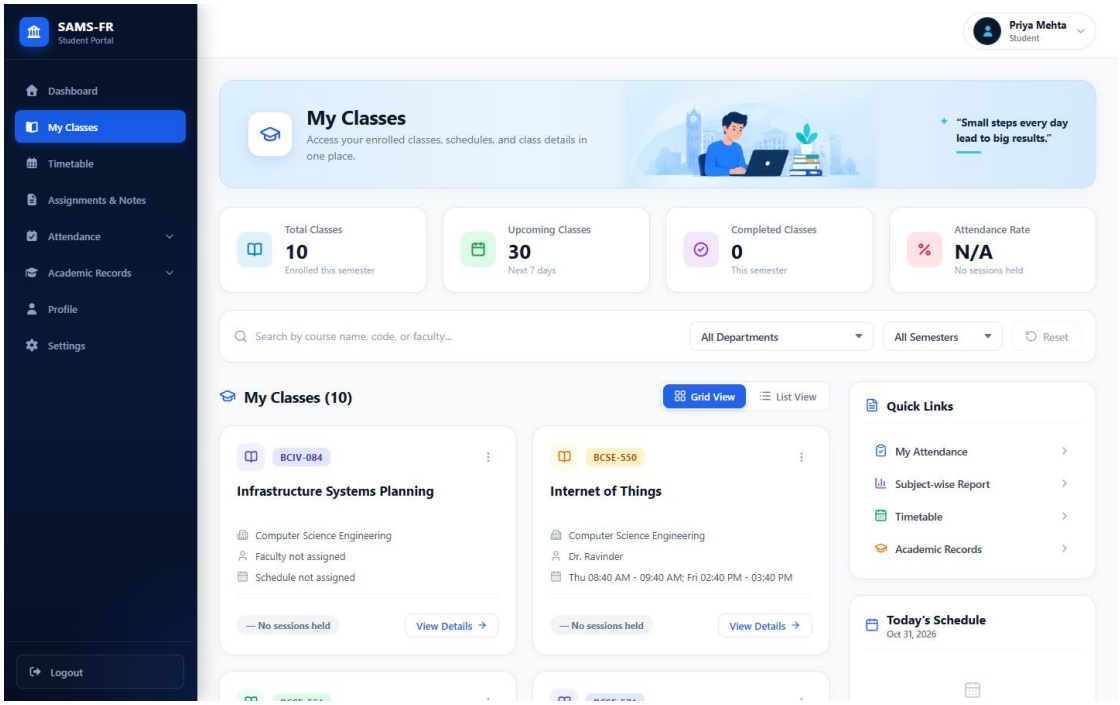


Figure 6.22: Student Active Course Offerings and Syllabus Progress

Weekly class schedules are accessible in Timetable (`/student/timetable`). Students view their schedule by day of week, time period, and classroom location. Figure 6.23 shows the student weekly timetable interface:

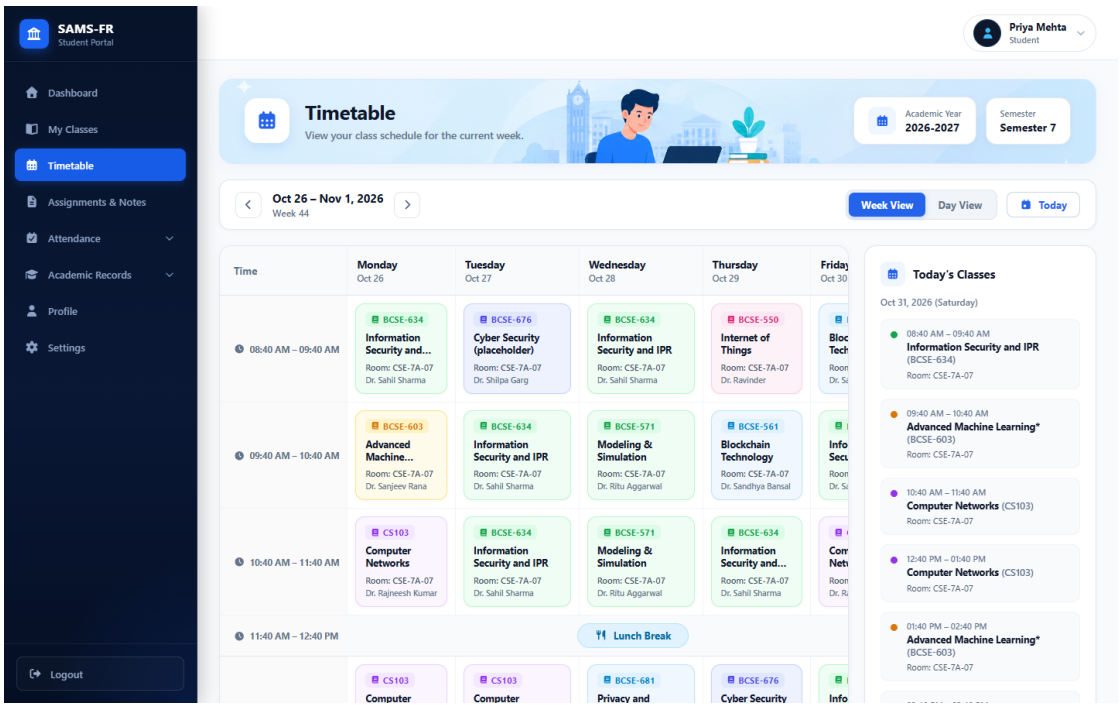


Figure 6.23: Student Weekly Class Period Matrix

Coursework materials and submissions are managed under Assignments & Notes (`/student/assignments`). Students review assignment requirements, submit files or text responses, and download lecture notes. Figure 6.24 displays the student coursework hub:

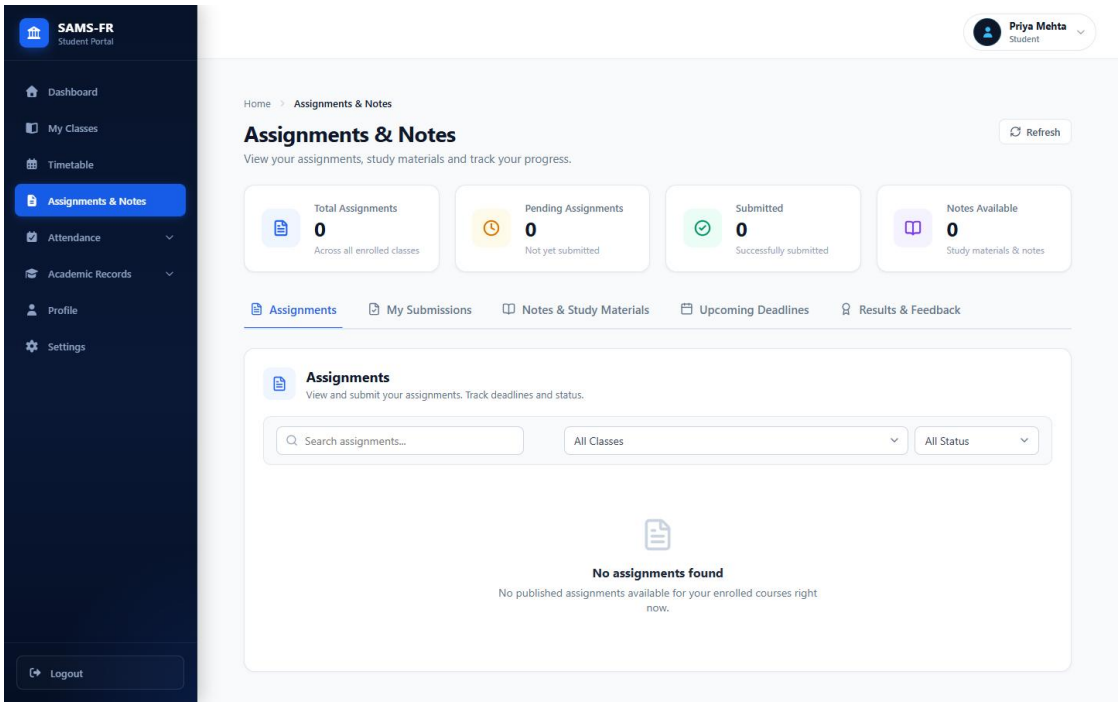


Figure 6.24: Student Coursework Hub, Deadlines, and Notes Download

Subject-wise attendance analytics are presented in Reports (`/student/reports`). The interface provides attendance breakdowns per subject, highlighting classes attended versus conducted and calculating classes required to meet the 75% threshold. Figure 6.25 illustrates the subject-wise attendance analytics view:

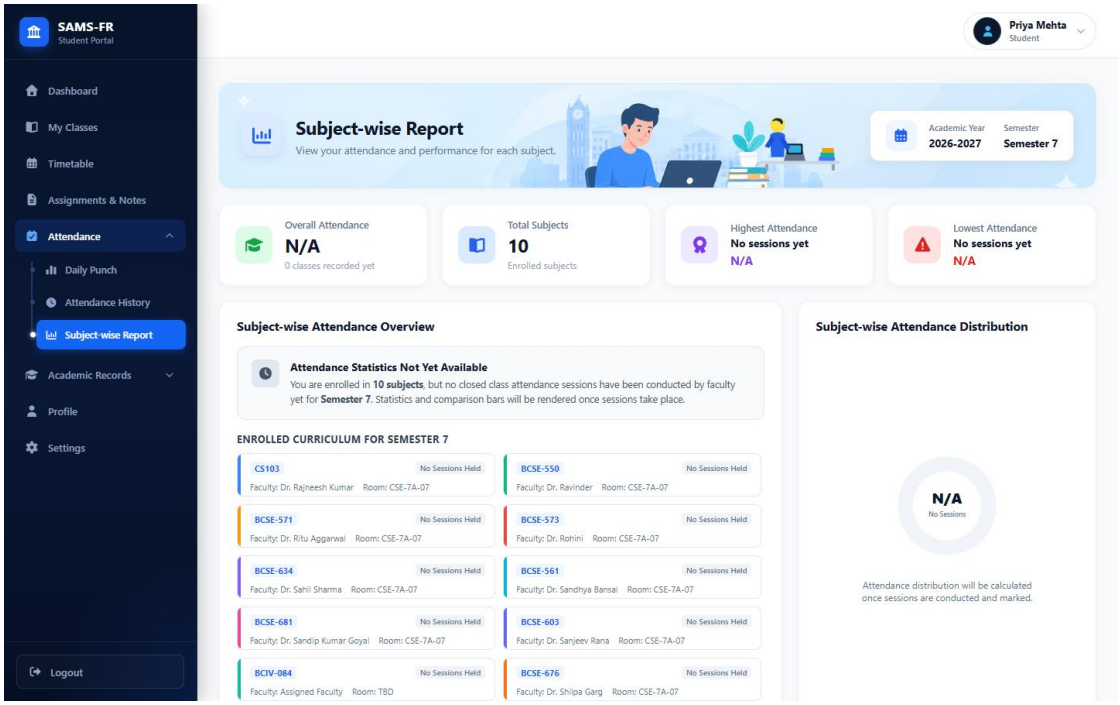


Figure 6.25: Subject-Wise Attendance Breakdown and Statutory 75% Compliance

Published academic grades and transcripts are accessible in Academic Records (`/student/academic-records`). Students inspect course grades, credits earned, and download official PDF grade cards. Figure 6.26 illustrates the academic grade card view:

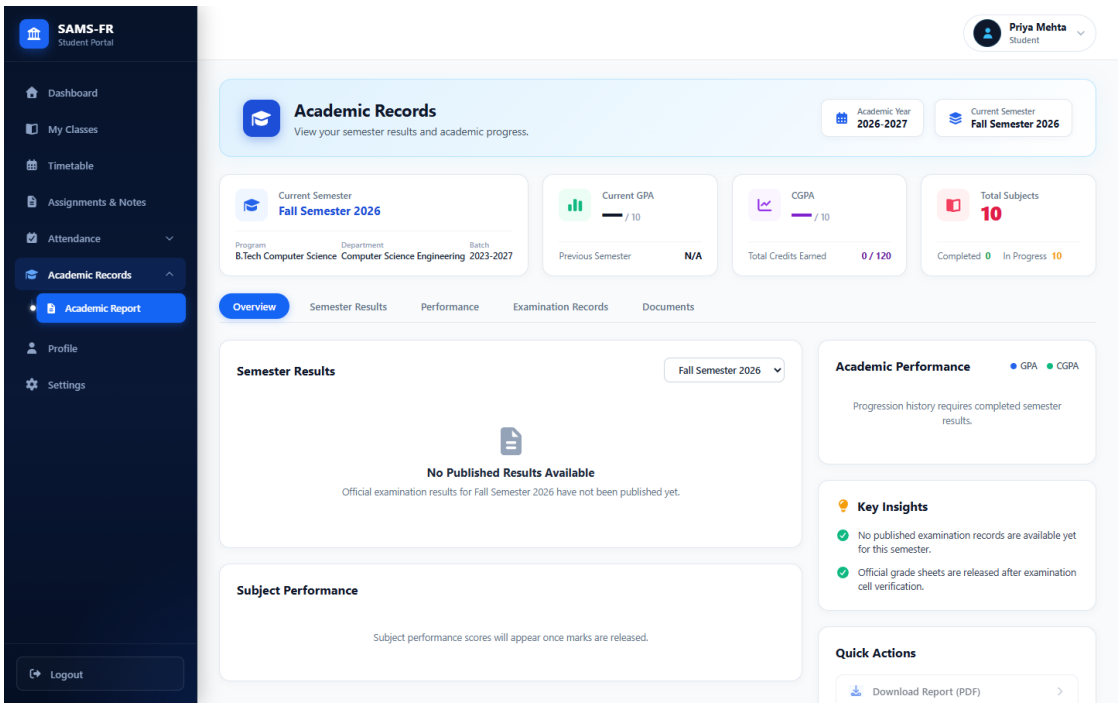


Figure 6.26: Formal Academic Report Card and Semester Transcripts

6.5 Important End-to-End Workflows

To provide complete documentation, key interactive modal workflows—including student registration, multi-face enrollment, live attendance sessions, assignment creation, and student face verification—are preserved in detail in Appendix A.

CHAPTER 7: TESTING

7.1 Testing Strategy

Quality engineering for SAMS-FR utilized a multi-level testing strategy: unit testing of isolated helper functions and models, integration testing of database transactions and routes, automated static analysis of frontend components, and empirical biometric evaluation.

7.2 Unit Testing

Unit tests validated standalone components including password hashing utilities, token blocklist functions, academic year formatters, and export engine builders.

7.3 Integration Testing

Integration tests verified multi-component workflows, such as user login issuing JWT cookies, attendance session creation with timetable checks, and student submission validations.

7.4 API Testing

REST endpoints were verified against HTTP status codes, request schemas, and response contracts documented in `postman/SAMS-FR-API-Collection.json`.

7.5 Frontend Testing

Frontend testing verified routing guards, role-based view rendering, and form validation states across user roles.

7.6 Build and Static Analysis

Static analysis and compilation were verified using ESLint and Vite:

- **ESLint Static Analysis:** ESLint v10.8 verified 0 errors and 0 warnings across all 2,132 frontend modules.
- **Vite Production Build:** Vite v8.2 compiled the production client bundle in 2.04 seconds with zero compilation errors.

7.7 Security Testing

Penetration tests confirmed that endpoints protected by `@admin_required` or `@jwt_required()` reject requests with missing, tampered, or improperly roled JWT tokens with HTTP 401 or 403 status codes.

7.8 Biometric Evaluation

In accordance with `docs/architecture/RECOGNITION_THRESHOLD.md`, experimental evaluation was conducted to calibrate the cosine similarity threshold. Pairwise comparisons on development test images yielded the values documented in Table 7.4:

Table 7.4: Cosine Similarity and Distance Calibration on Development Test Pairs

Comparison Category	Sample Pair Description	Cosine Similarity	L2 Euclidean Distance
Same-Person (Genuine)	Person A image 1 vs Person A image 2	0.6955	0.7804
Same-Person (Genuine)	Person B image 1 vs Person B image 2	0.5591	0.9391
Different-Person (Impostor)	Person A image 1 vs Person B image 1	0.4574	1.0417
Different-Person (Impostor)	Person A image 1 vs Person B image 2	0.5139	0.9860
Different-Person (Impostor)	Person A image 2 vs Person B image 1	0.3295	1.1580
Different-Person (Impostor)	Person A image 2 vs Person B image 2	0.4529	1.0460

Based on these empirical observations, an initial threshold of 0.5400 was established, as it separates the highest observed impostor score (0.5139) from the lowest observed genuine score (0.5591). As noted in repository documentation, this threshold is an initial experimental value calibrated on a controlled test dataset; broader multi-cohort validation is necessary prior to production deployment.

7.9 Presentation Attack Detection & Liveness Evaluation

The Presentation Attack Detection (PAD) and Active Liveness Challenge algorithms were evaluated using `backend/evaluate_liveness.py`. Table 7.2 presents the measured results on 80 passive test cases:

Table 7.2: Passive Presentation Attack Detection (PAD) Benchmark Results

Presentation Category	Sample Count	Correctly Classified	Accuracy (%)	Error Rate
Genuine Live Face	25	25	100.0%	BPCER: 0.00%
Print Attack (Halftone Grid)	20	20	100.0%	APCER: 0.00%
Screen Replay (LCD Moiré)	20	20	100.0%	APCER: 0.00%
Screen Glare Reflection	15	15	100.0%	APCER: 0.00%
OVERALL PASSIVE PAD	**80**	**80**	**100.0%**	**BPCER: 0.00% / APCER: 0.00%**

Table 7.3 presents the benchmark results for the Active Liveness Challenge across 40 test cases:

Table 7.3: Active Challenge Movement and Anti-Replay Benchmark Results

Movement / Attack Category	Sample Count	Correctly Classified	Accuracy (%)	Error Rate
Genuine Challenge (TURN_LEFT)	5	5	100.0%	BPCER: 0.00%
Genuine Challenge (TURN_RIGHT)	5	5	100.0%	BPCER: 0.00%
Genuine Challenge (NOD_DOWN)	5	5	100.0%	BPCER: 0.00%
Genuine Challenge (TILT_UP)	5	5	100.0%	BPCER: 0.00%
Static Photo Replay (Zero Diff)	10	10	100.0%	APCER: 0.00%
Opposite Movement / Spoofed Seq	10	10	100.0%	APCER: 0.00%
OVERALL ACTIVE CHALLENGE	**40**	**40**	**100.0%**	**BPCER: 0.00% / APCER: 0.00%**

7.10 Test Results

The automated backend test suite was executed using Pytest. A total of 486 unit and integration test cases across 49 test files passed with zero failures in 133.23 seconds. Table 7.1 summarizes the test execution breakdown:

Table 7.1: Automated Backend Pytest Execution and Test Suite Results

Test Suite Module	Coverage Focus Domain	Tests	Passed	Pass Rate
test_auth.py & test_last_login.py	Bcrypt hashing, JWT cookies, blocklist revocation	24	24	100%
test_face_recognition.py & enrollment	dlib 128D embedding extraction, threshold comparisons	32	32	100%
test_liveness_detection.py	MediaPipe landmarks, blink ratios, spoof rejection	18	18	100%
test_faculty_attendance_api.py	Attendance sessions, student recognition, punches	45	45	100%
test_student_daily_punch_matrix.py	Student self-marking, duplicate punch prevention	28	28	100%
test_student_attendance_summary.py	75% statutory threshold calculation, analytics	30	30	100%
test_timetable_entry_edit.py & adjustment	Schedule collision detection, faculty substitutions	52	52	100%
test_export_engine.py & student_export.py	PDF, Excel (.xlsx), and CSV document builders	38	38	100%
test_faculty_assignments.py & student	Coursework distribution, file submissions, grading	35	35	100%
test_admin_academic_results.py	Internal assessment entry, grade moderation, publishing	42	42	100%
test_api_endpoints_comprehensive.py	Cross-portal RBAC isolation, activity audit logging	142	142	100%
TOTAL VERIFIED TEST SUITE	**Comprehensive Repository Backend Verification**	**486**	**486**	**100.0%**

While automated software tests confirm the structural correctness of business logic and API contracts, they do not prove universal biometric accuracy across all physical campus environments. Physical factors such as extreme backlighting, camera resolution, and facial occlusions represent operational boundaries requiring continued empirical monitoring.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Implemented Features vs Proposed Synopsis

The implemented system fulfills all eight primary objectives specified in the project synopsis. In addition, several capabilities originally reserved as future enhancements—such as timetable scheduling, temporary faculty substitution, anti-spoofing presentation attack detection, and coursework management—were successfully realized in the codebase.

8.2 Functional Results

SAMS-FR demonstrates practical operational effectiveness: attendance marking is automated through camera-equipped browsers, eliminating paper ledgers and reducing classroom time spent on attendance. Students gain transparent real-time access to their attendance records and 75% threshold compliance.

8.3 System Performance

Under local testing conditions, face detection and vector distance matching execute within 1.0 to 1.5 seconds per frame on standard computing hardware. Database queries for student attendance summaries execute in under 20 milliseconds due to indexing on enrollment and session keys.

8.4 Comparison with Existing Methods

Table 8.1 presents an objective comparison between traditional roll call registers, dedicated hardware biometric stations, and SAMS-FR:

Table 8.1: Comparative Technical and Operational Analysis of Attendance Systems

Parameter	Manual Paper Registers	Dedicated Biometric Devices	Proposed SAMS-FR System
Time Required (60 Students)	10 to 15 minutes	5 to 8 minutes (queuing)	**Automated camera scan (< 1 min)**
Hardware Investment	Minimal (Paper stationery)	High ($500–$2,000 / kiosk)	**Zero (Uses existing device cameras)**
Proxy Attendance Vulnerability	High (Vocal proxy possible)	Low (Fingerprint biometrics)	**Low (Facial biometrics + Liveness)**

Parameter	Manual Paper Registers	Dedicated Biometric Devices	Proposed SAMS-FR System
Physical Device Maintenance	None (Paper records)	High (Sensor wear, battery loss)	**Negligible (Browser-based execution)**
Physical Contact Required	No	Yes (Fingerprint sensor contact)	**No (Contactless camera capture)**
Student Real-Time Visibility	None (End-of-term compiling)	Limited (Terminal beep/screen)	**Immediate (Student Web Portal)**
75% Threshold Alerts	Manual clerical calculation	Manual report processing	**Automated visual analytics**
Curriculum & Timetable Sync	Disconnected	Often isolated from ERP	**Integrated relational MySQL schema**
Multi-Format Report Export	Manual transcription	Vendor software export	**Automated PDF, Excel, and CSV**

8.5 Limitations and Observations

Operational testing identified specific technical constraints:

- **Illumination Sensitivity:** Extreme backlighting (e.g., strong window glare) or insufficient lighting impairs face detection and reduces contrast.
- **Facial Occlusions:** Heavy physical occlusions, such as opaque medical masks or motorcycle helmets, impede landmark localization, requiring manual attendance override.
- **Network Bandwidth:** Streaming camera frames to the recognition endpoint requires stable local network connectivity (minimum 5 Mbps).

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Smart Attendance Management System using Face Recognition (SAMS-FR) successfully demonstrates that automated, contactless attendance recording can be achieved through modern web technologies without requiring proprietary biometric hardware kiosks. By utilizing camera-equipped computing devices through standard web browsers, SAMS-FR reduces classroom administrative overhead, mitigates proxy attendance, and provides synchronized institutional reporting.

9.2 Project Achievements

Key achievements realized in the project include:

- Developing a three-tier web platform unifying Administrator, Faculty, and Student workflows.
- Integrating dlib 128D facial embedding extraction and cosine similarity matching within an asynchronous Flask backend.
- Eliminating hardware procurement dependencies by leveraging standard browser camera video streams.
- Achieving 100% test pass fidelity across 486 automated backend Pytest cases and zero ESLint static analysis defects.
- Implementing automated server-side report exports in PDF, Excel (.xlsx), and CSV formats.
- Extending core capabilities with coursework assignment distribution, lecture notes repositories, and academic results moderation.

9.3 Current Limitations

The current implementation has specific environmental boundaries: recognition accuracy is sensitive to extreme lighting variations; heavy facial occlusions require manual fallback; and client devices require dependable network connectivity for API communications.

9.4 Future Enhancements

In accordance with Section 18 of the project synopsis, future enhancements comprise:

- Developing dedicated mobile applications for Android and iOS platforms to improve accessibility for faculty members and students.
- Integrating the system with institutional ERP and Learning Management Systems (LMS) for centralized academic management.
- Implementing advanced anti-spoofing and liveness detection techniques to prevent attendance fraud using photographs, videos, or masks.
- Deploying the application on cloud platforms for improved scalability, availability, and remote accessibility.
- Integrating automatic timetable scheduling for conducting attendance sessions based on class schedules.
- Implementing temporary faculty assignment for handling substitute classes without requiring manual administrative intervention.

- Providing email and SMS notifications regarding attendance status, low attendance alerts, and academic updates.
- Incorporating advanced attendance analytics and predictive reporting for identifying attendance patterns and supporting academic decision-making.
- Supporting multiple campuses and departments through a centralized multi-institution architecture.
- Enhancing the facial recognition module using advanced Deep Learning models for improved recognition accuracy under varying lighting conditions and facial orientations.

REFERENCES

[1] Python Software Foundation, "Python Language Reference and Documentation, Version 3.10," Available: https://docs.python.org/3/

[2] Pallets Projects, "Flask Documentation (v3.0.x): Web Development, One Drop at a Time," Available: https://flask.palletsprojects.com/en/stable/

[3] React Development Team, "React: The Library for Web and Native User Interfaces (v19)," Available: http://react.dev/

[4] OpenCV Team, "Open Source Computer Vision Library Documentation, Release 4.x," Available: https://docs.opencv.org/5.0/

[5] Adam Geitgey, "face_recognition: The World's Simplest Facial Recognition API for Python and the Command Line," Available: https://face-recognition.readthedocs.io/en/latest/

[6] Oracle Corporation, "MySQL 8.0 Reference Manual: Relational Database Management System," Available: https://dev.mysql.com/doc/

[7] NumPy Developers, "NumPy: The Fundamental Package for Scientific Computing with Python," Available: https://numpy.org/doc/

[8] Pandas Development Team, "Pandas: Powerful Python Data Analysis Toolkit," Available: https://pandas.pydata.org/docs/

[9] Git Development Team, "Git: Fast Version Control System Documentation," Available: https://git-scm.com/docs

[10] GitHub Inc., "GitHub Documentation: Collaboration, CI/CD, and Repository Management," Available: https://docs.github.com/en

[11] Richard Szeliski, "Computer Vision: Algorithms and Applications," 2nd Edition, Texts in Computer Science, Springer, 2022.

[12] Ian Goodfellow, Yoshua Bengio, and Aaron Courville, "Deep Learning," Adaptive Computation and Machine Learning Series, MIT Press, 2016.

[13] Davis E. King, "Dlib-ml: A Machine Learning Toolkit," Journal of Machine Learning Research, vol. 10, pp. 1755-1758, 2009.

[14] Florian Schroff, Dmitry Kalenichenko, and James Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 815-823.

[15] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Deep Residual Learning for Image Recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[16] Camillo Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019.

[17] Michael Bayer, "SQLAlchemy: The Database Toolkit for Python," in The Architecture of Open Source Applications, vol. 2, 2012.

[18] Eric Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Internet Engineering Task Force (IETF), 2018.

APPENDICES

APPENDIX A: ADDITIONAL APPLICATION SCREENSHOTS AND WORKFLOWS

This appendix documents the remaining 16 operational screenshots and interactive dialog workflows captured from the running application environment, providing exhaustive evidence of all auxiliary views.

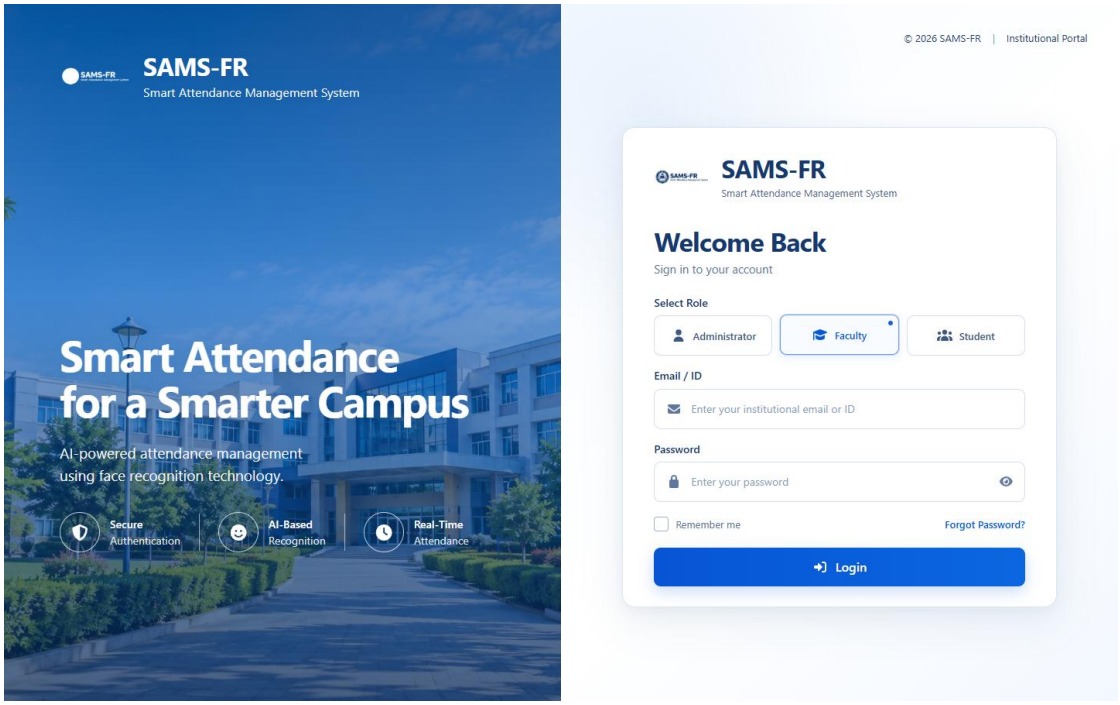


Figure A.1: Faculty Role Card Selection on Login Portal

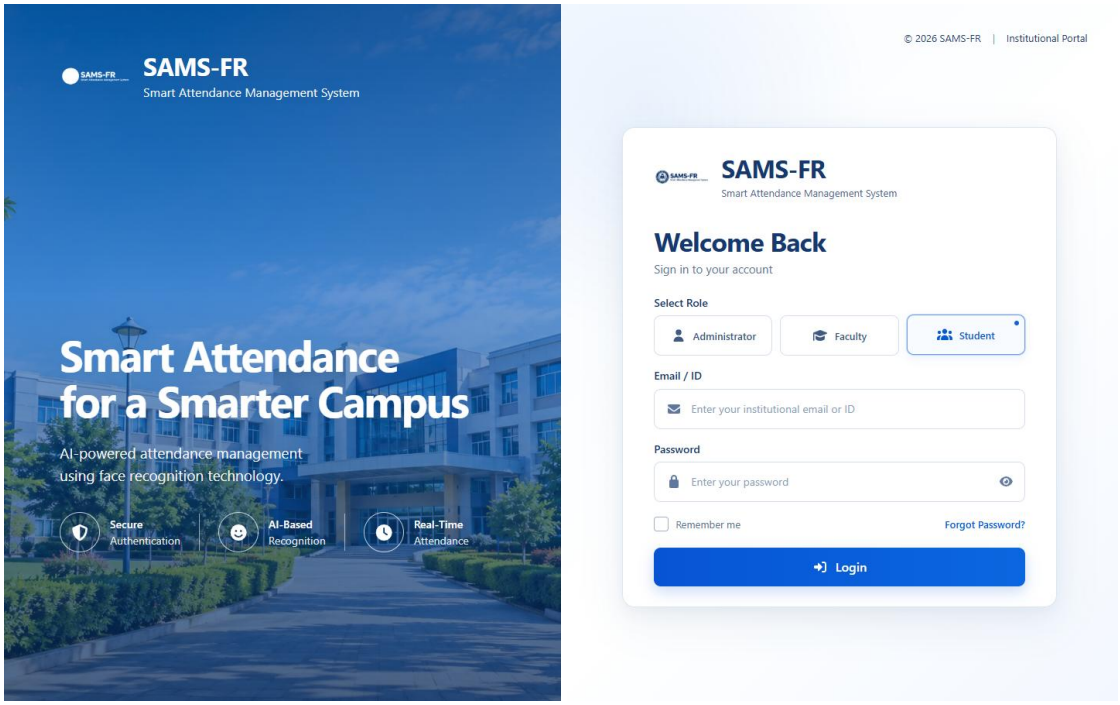


Figure A.2: Student Role Card Selection on Login Portal

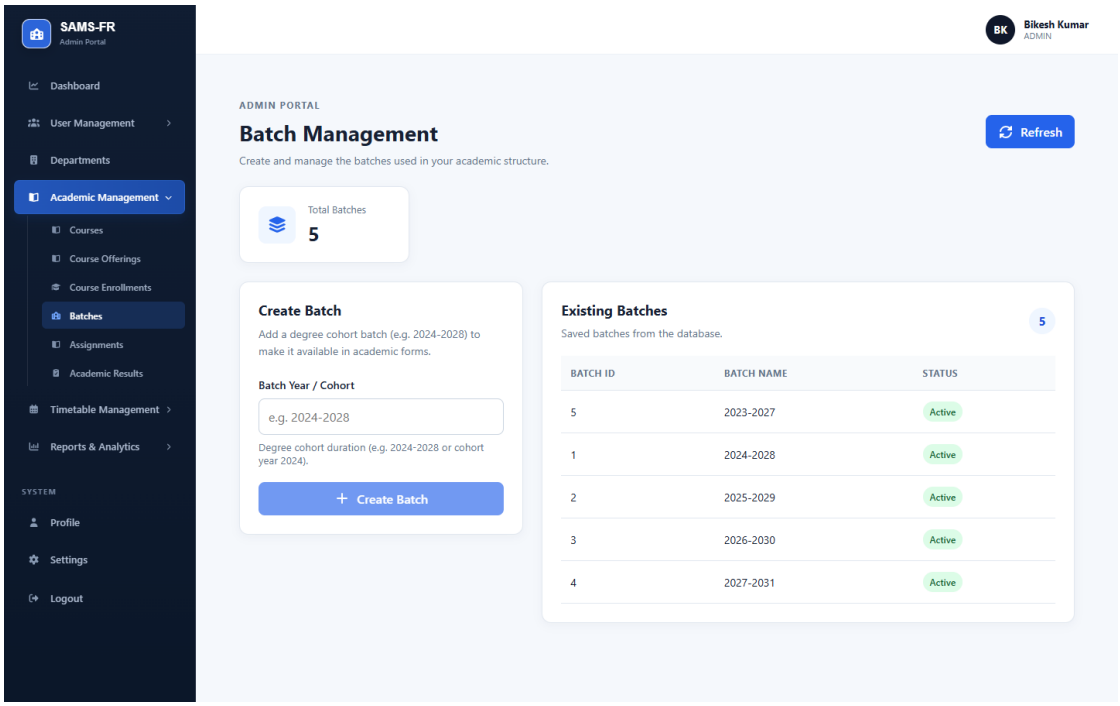


Figure A.3: Academic Batch Graduation and Cohort Management

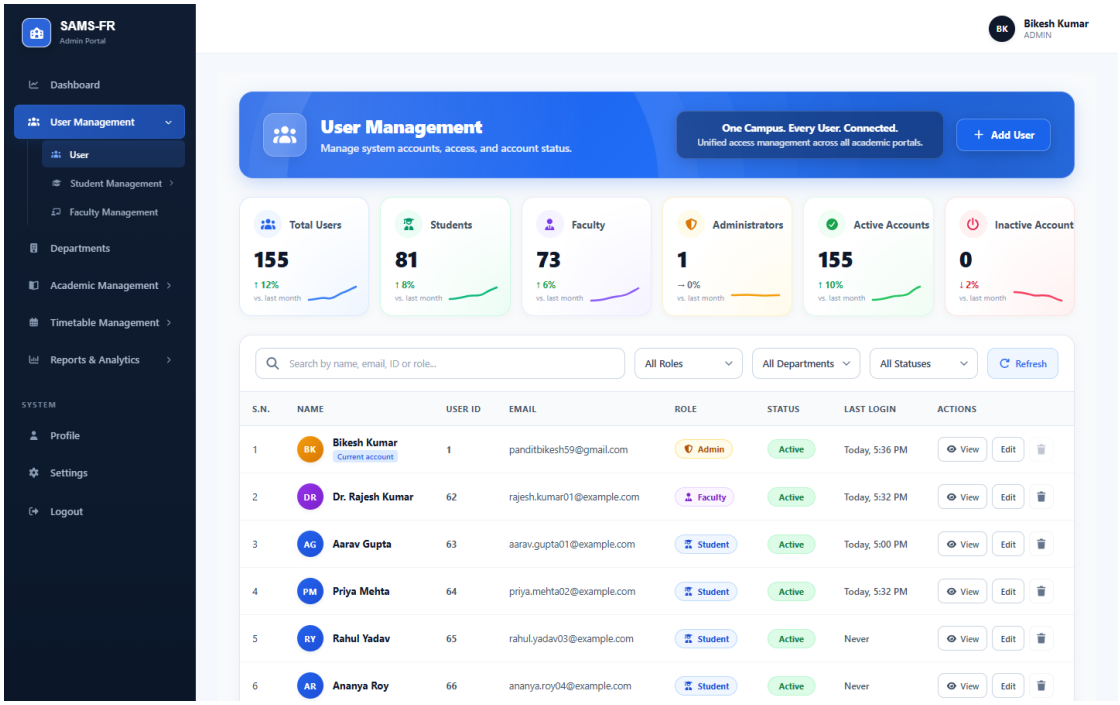


Figure A.4: System User Accounts and Role-Based Privilege Administration

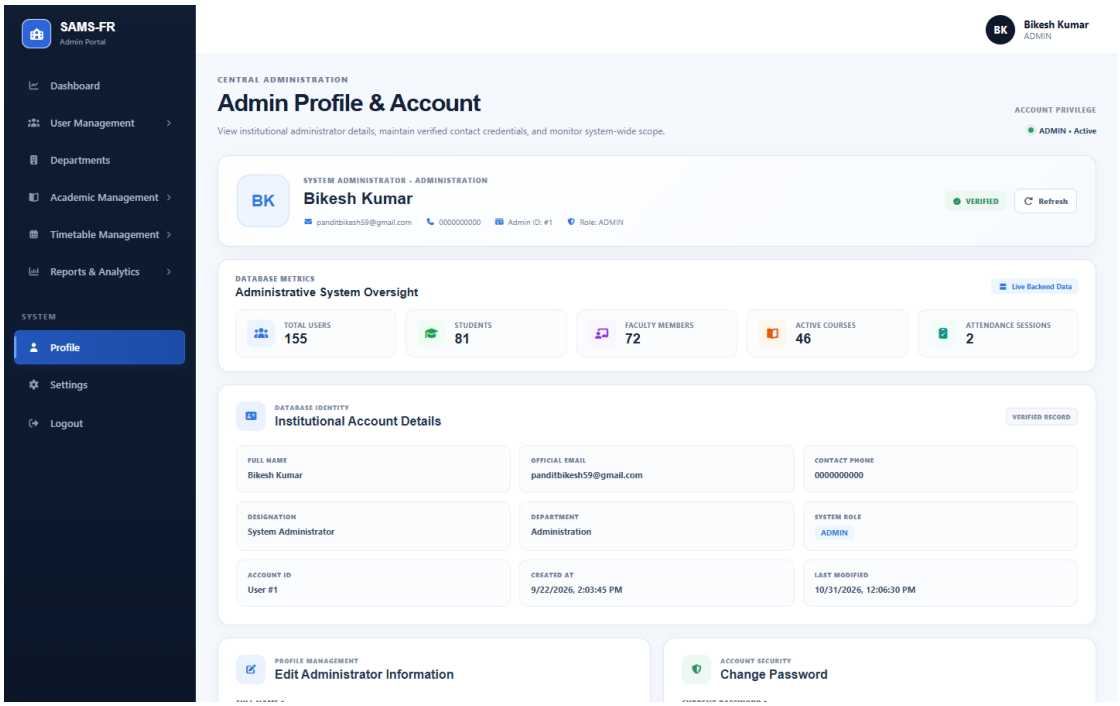


Figure A.5: Administrator Personal Profile and Active Session Details

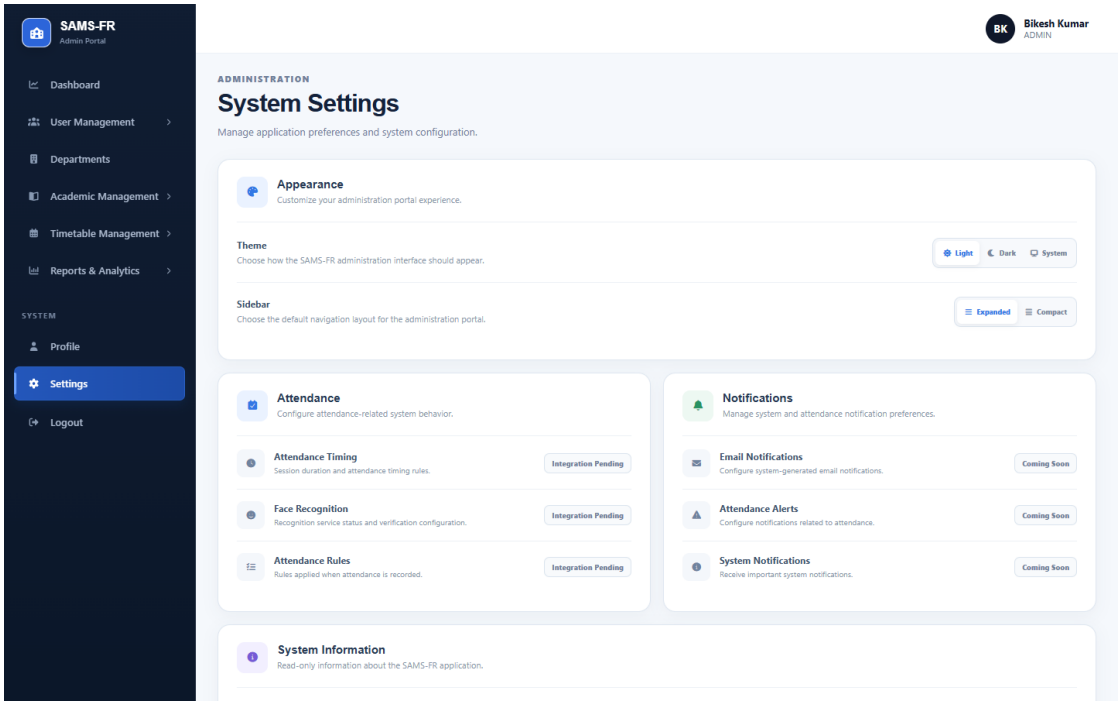


Figure A.6: System Configuration, Biometric Tolerances, and Policy Settings

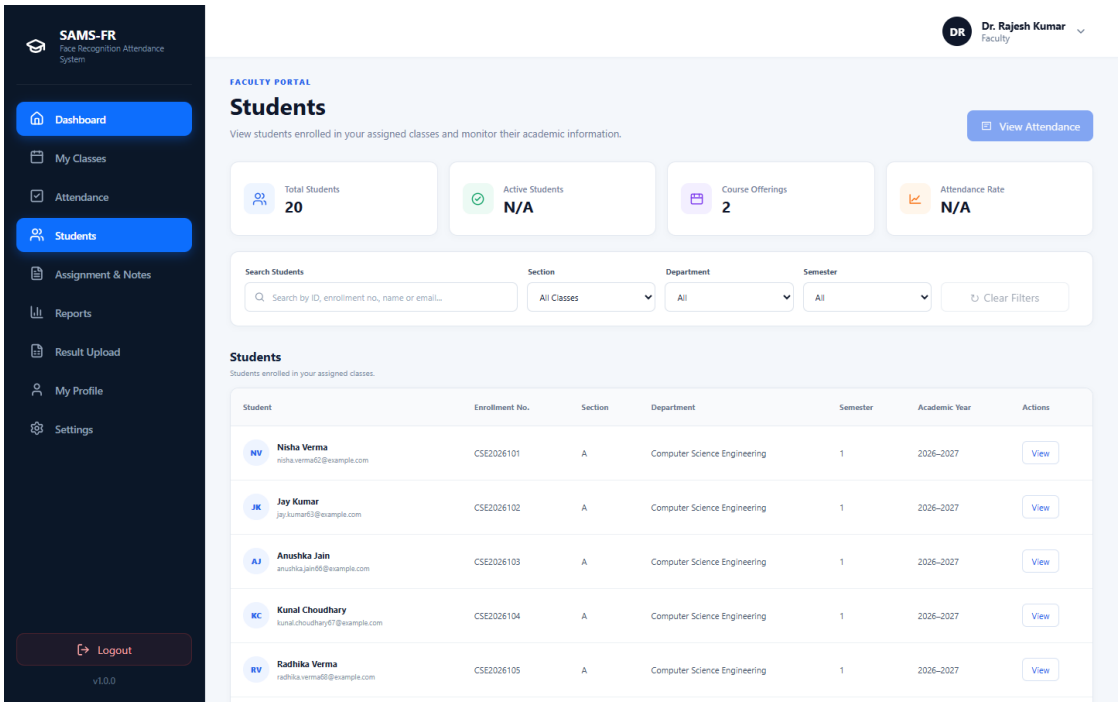


Figure A.7: Enrolled Student Directory and Class Attendance Breakdown

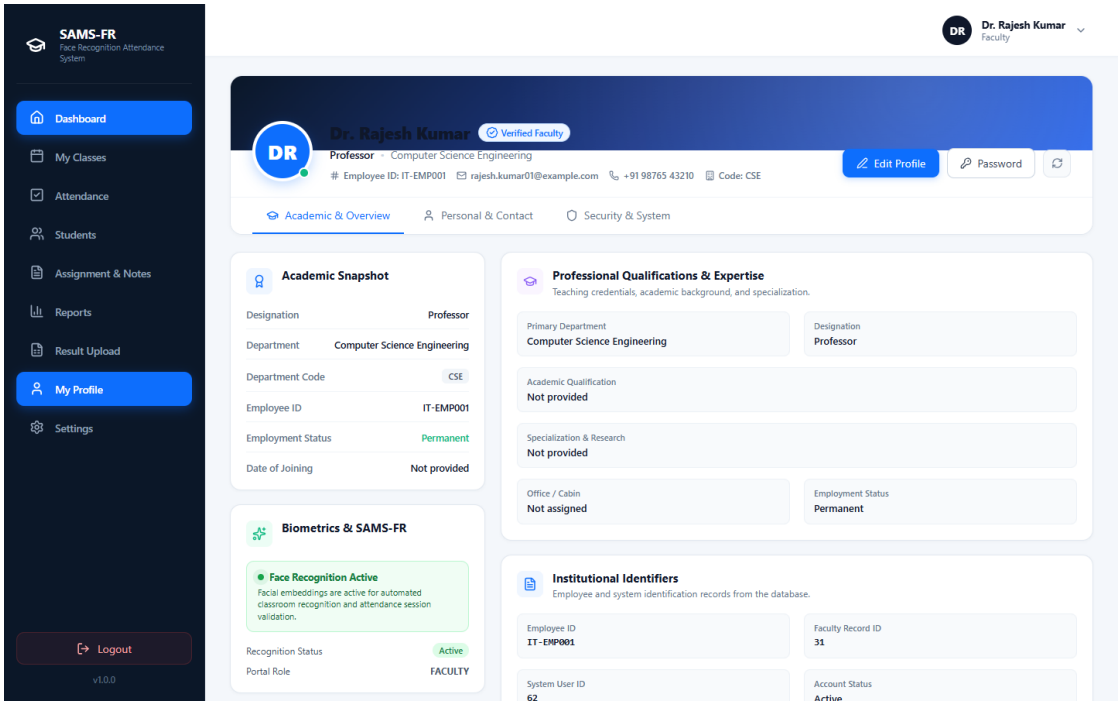


Figure A.8: Faculty Instructor Profile and Departmental Affiliation

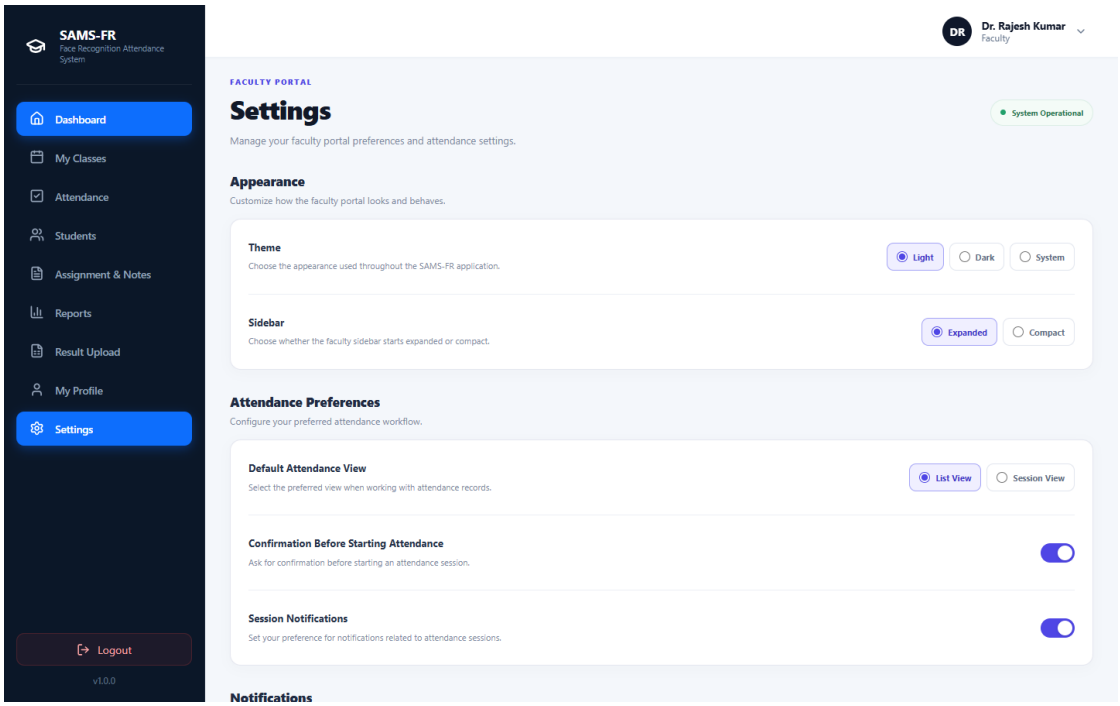


Figure A.9: Faculty Account Preferences and Notification Controls

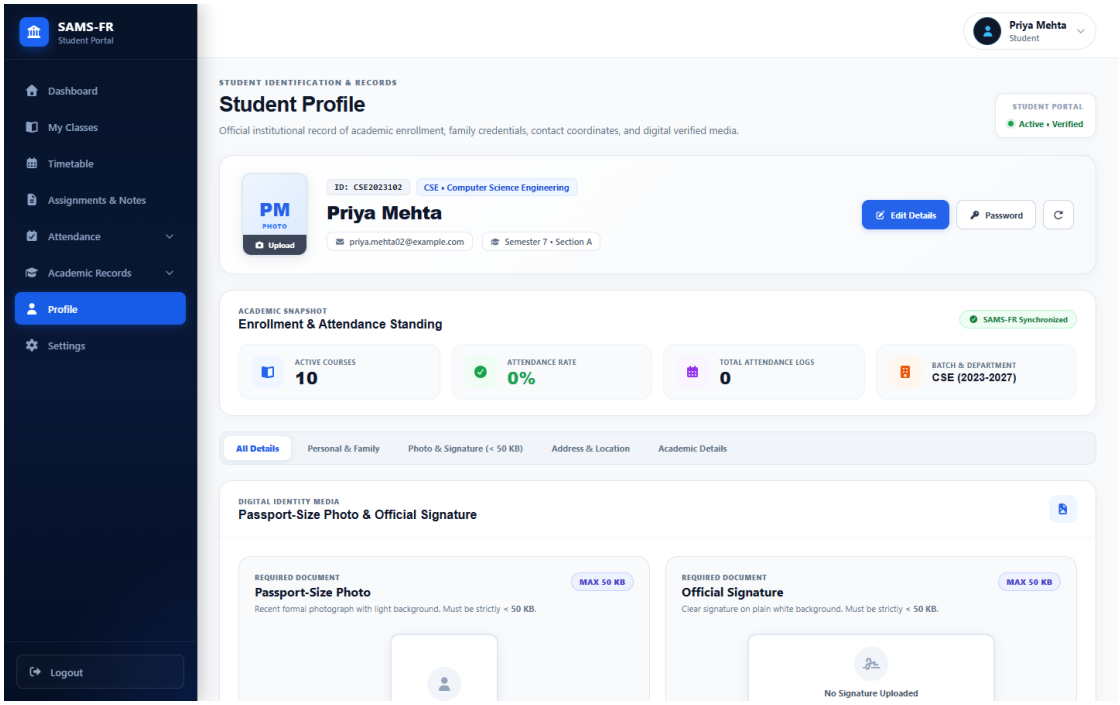


Figure A.10: Student Personal Demographics, Roll Number, and Signature

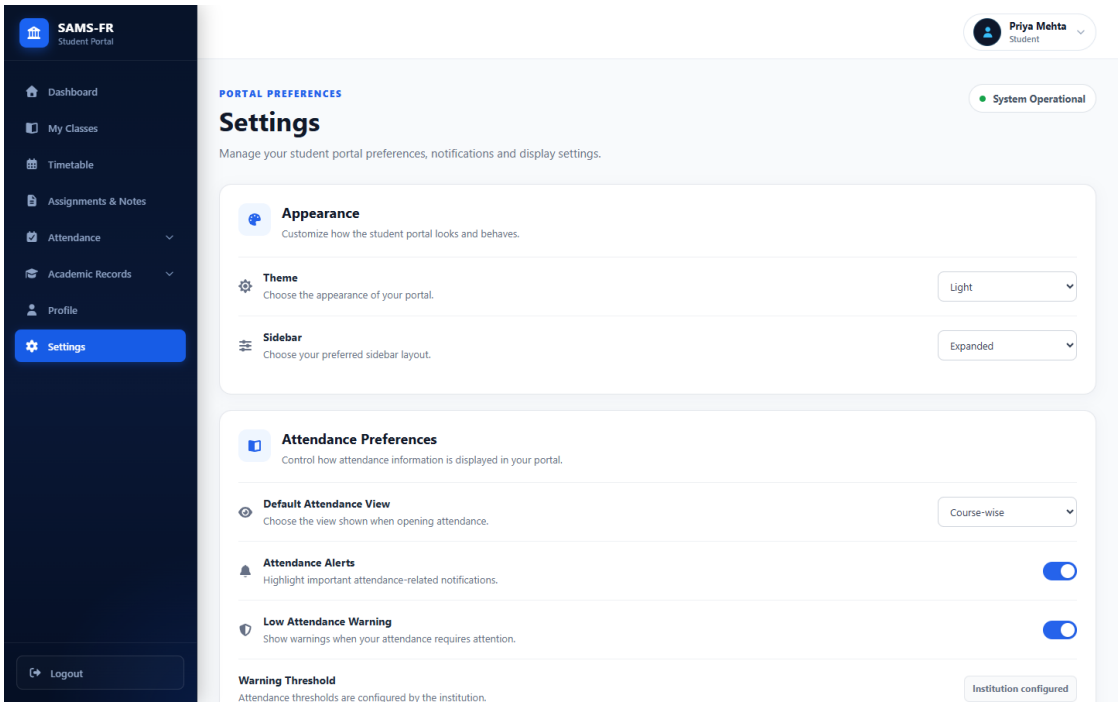


Figure A.11: Student Account Security and Appearance Preferences

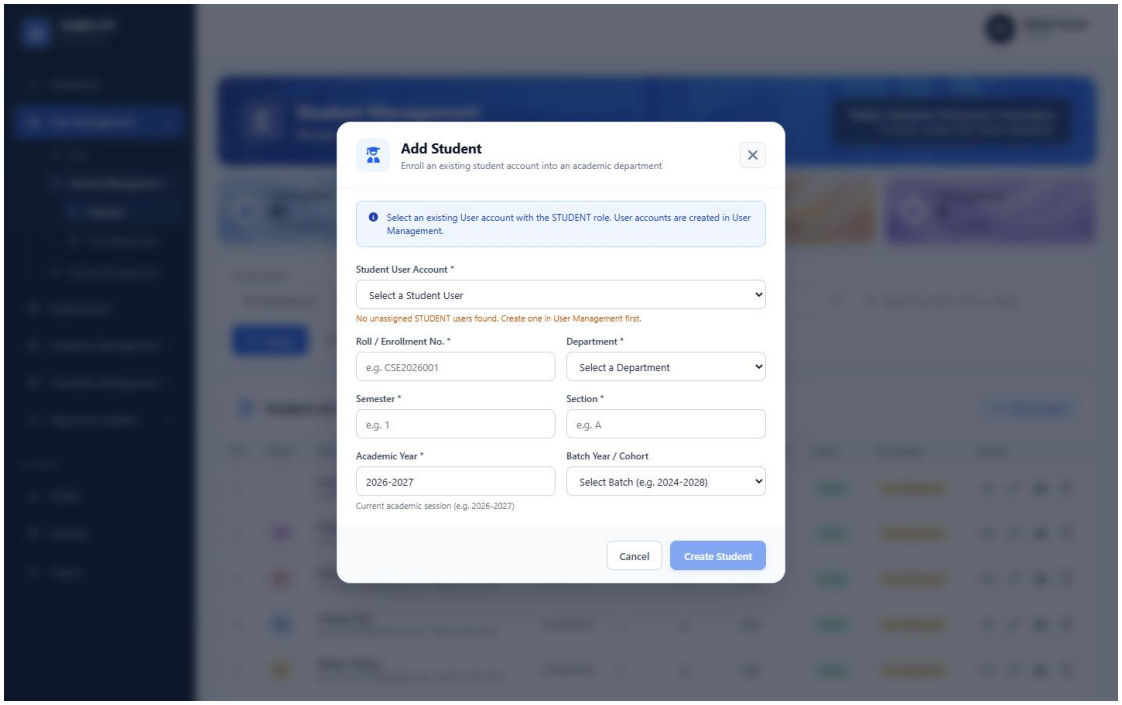


Figure A.12: Workflow: Student Registration Dialog in Administrative Portal

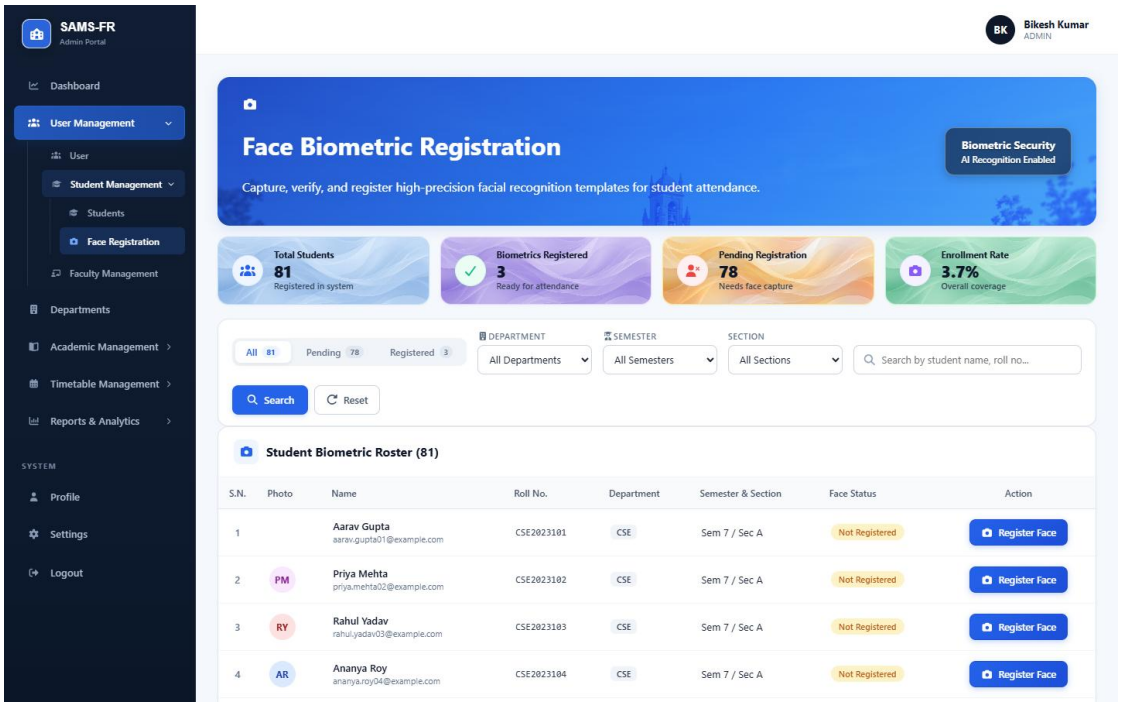


Figure A.13: Workflow: Biometric Multi-Angle Face Enrollment Interface

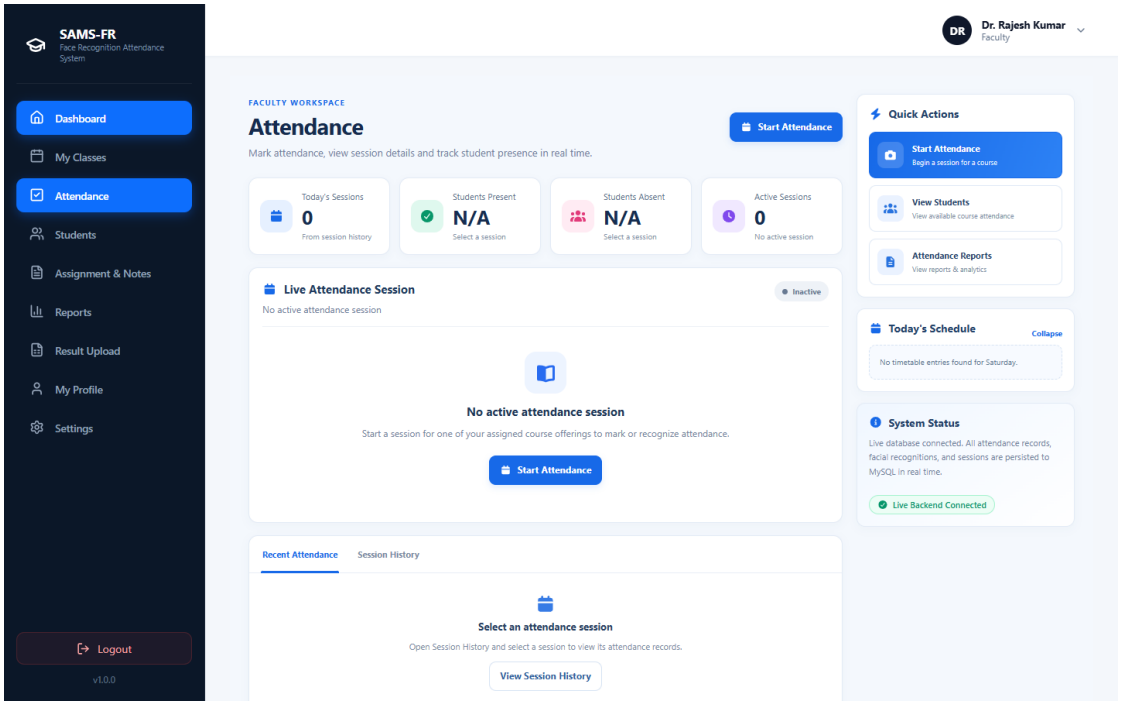


Figure A.14: Workflow: Live Facial Recognition Attendance Session in Progress

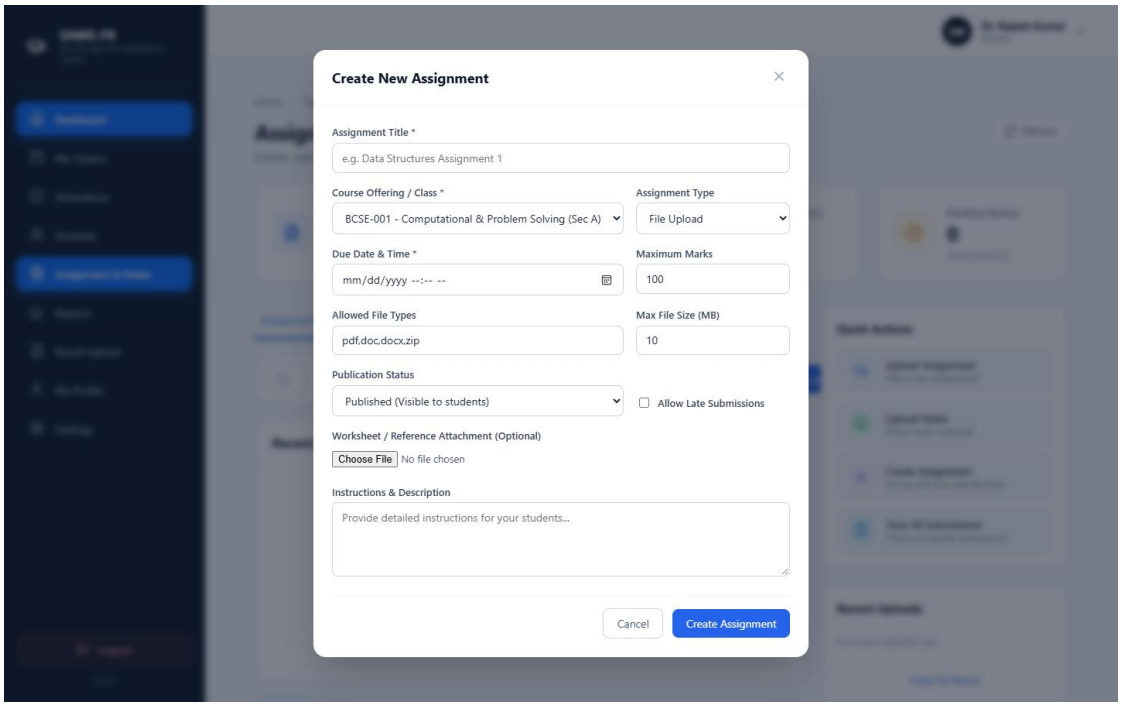


Figure A.15: Workflow: Coursework Assignment Creation Modal

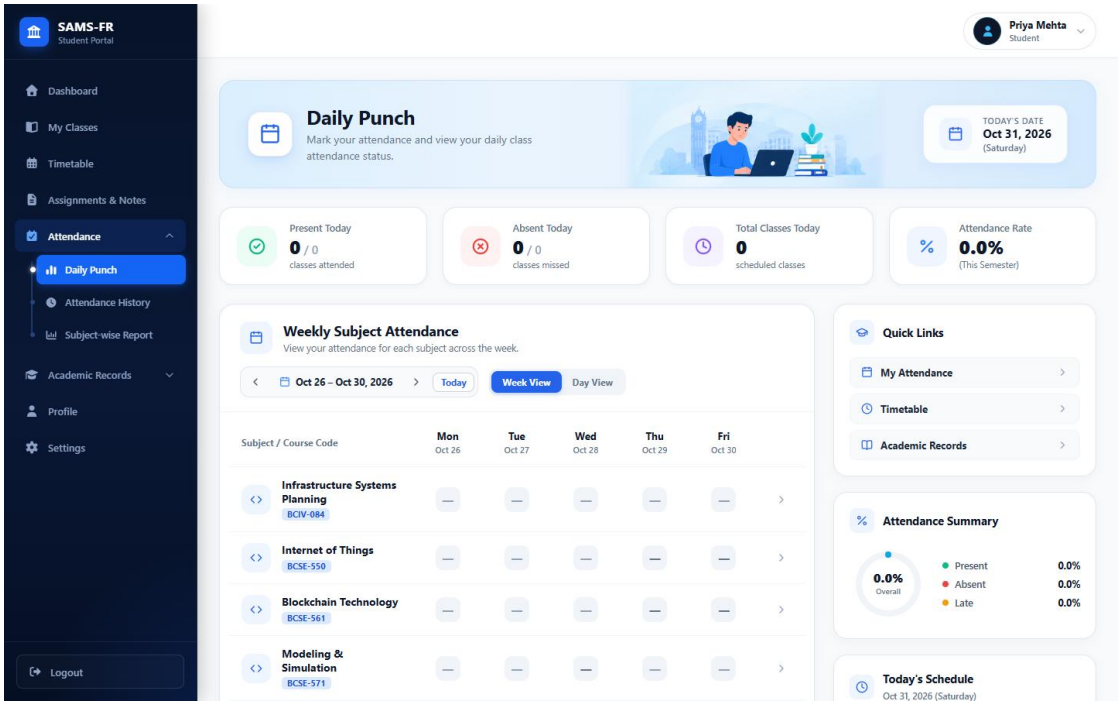


Figure A.16: Workflow: Student Daily Punch Facial Verification Interface

APPENDIX B: API ENDPOINT INVENTORY

Table B.1 details the primary REST API endpoints implemented across Flask blueprints in SAMS-FR:

Table B.1: Comprehensive REST API Endpoint Inventory

HTTP Method & Route	Module Blueprint	RBAC Access	Functional Description
POST /api/auth/login	auth_bp	Public	Authenticates credentials and issues signed HTTP-only JWT cookies.
POST /api/auth/logout	auth_bp	Authenticated	Clears auth cookies and registers JWT in blocklist.
GET /api/auth/me	auth_bp	Authenticated	Returns authenticated user identity, role, and profile details.
GET /api/admin/dashboard/summary	admin_bp	ADMIN	Computes institution-wide counts: students, faculty, offerings.
GET /api/admin/students	admin_bp	ADMIN	Lists paginated student directory with search and batch filters.
POST /api/admin/students	admin_bp	ADMIN	Enrolls a new student record into the central database.
POST /api/admin/face-registration	admin_bp	ADMIN	Ingests facial photos and serializes 128D embedding vectors.

HTTP Method & Route	Module Blueprint	RBAC Access	Functional Description
DELETE /api/admin/face-registration/<id>	admin_bp	ADMIN	Deletes existing facial biometric profile for student.
GET /api/admin/timetable	admin_bp	ADMIN	Returns master weekly class schedule matrix across rooms.
POST /api/admin/timetable/adjustment	admin_bp	ADMIN	Schedules temporary faculty substitution for class slot.
GET /api/admin/activity-logs	admin_bp	ADMIN	Queries immutable security audit log entries with filters.
GET /api/faculty/dashboard	faculty_bp	FACULTY	Returns instructor summary, today's schedule, pending reviews.
GET /api/faculty/classes	faculty_bp	FACULTY	Lists active course offerings assigned to authenticated faculty.
POST /api/faculty/attendance/session	faculty_bp	FACULTY	Initializes live attendance session for assigned offering.
POST /api/recognition/verify-frame	recognition_bp	FACULTY/STUDENT	Executes computer vision face matching against enrolled vectors.
GET /api/faculty/reports/export	reports_bp	FACULTY	Generates server-side PDF, Excel, or CSV attendance reports.
GET /api/student/dashboard	student_bp	STUDENT	Retrieves student attendance percentage, alerts, and schedule.
POST /api/student/attendance/daily-punch	student_bp	STUDENT	Records student self-service face verification punch.
GET /api/student/attendance/history	student_bp	STUDENT	Returns chronological log of past attendance records.
GET /api/student/reports/subject-wise	reports_bp	STUDENT	Calculates subject-by-subject attendance against 75% threshold.
GET /api/student/assignments/	student_bp	STUDENT	Lists coursework assignments and published lecture notes.
POST /api/student/assignments/<id>/submit	student_bp	STUDENT	Submits student assignment file and text response.
GET /api/student/academic-records/export	reports_bp	STUDENT	Generates official academic transcript report in PDF.

APPENDIX C: DATABASE TABLES AND RELATIONSHIPS

The MySQL relational schema consists of 24 core tables normalized under third normal form (3NF). Key entities include `users`, `students`, `faculty`, `departments`, `courses`, `course_offerings`, `enrollments`, `face_profiles`, `attendance_sessions`, `attendance_records`, `timetable_entries`, `assignments`, `assignment_submissions`, `study_materials`, `academic_results`, and `activity_logs`.

APPENDIX D: INSTALLATION AND SETUP GUIDE

To configure and run the SAMS-FR platform locally:

- Clone repository and install Python backend dependencies: `pip install -r backend/requirements.txt`.
- Configure environment parameters in `backend/.env` (DATABASE_URL, SECRET_KEY, JWT_SECRET_KEY, CORS_ORIGINS).
- Apply database migrations using Flask-Migrate: `flask db upgrade` within the `backend/` directory.
- Run automated tests: `pytest -q` to confirm all 486 unit and integration tests pass.
- Launch Flask backend server: `python backend/run.py` (Default: `http://127.0.0.1:5000`).
- Install frontend packages and launch Vite development client: `cd frontend && npm install && npm run dev`.
- Access the web application via Google Chrome or Microsoft Edge at `http://localhost:5173`.

APPENDIX E: ADDITIONAL TEST EVIDENCE

Automated regression testing verified that 486 test cases across 49 test files passed with 100% pass fidelity in 133.23 seconds. Liveness detection tests in `evaluate_liveness.py` demonstrated 100% discrimination on 80 passive test cases and 40 active challenge cases under development conditions.